INTRODUCTION TO WEB TECHNOLOGIES

AUE-CSC 204



Course Outline

- **Departments:** Computer Science, Cybersecurity & Software Engineering
- **Semester/Session**: 2nd Semester 2024/2025
- **Course code/title**: AUE-CSC 204; Introduction to web technologies (3 units)
- **Course prerequisite**: None
  Course Lecturer: Akintunde Adedamola E.
- **Email**: akintunde.adedamola@adelekeuniversity.edu.ng
- **Tel**: +2348073373591

Learning Objectives

1. **Understand key web technologies**: Identify and describe the key technologies and programming models of the web.
2. **Client-side technologies**: Explain and apply client-side technologies such as HTML, CSS, and JavaScript.
3. **Content formats and creation**: Understand and create content using client-side technologies.
4. **Server-side technologies**: Describe and explain server-side technologies such as web and application servers, database back ends, and client-server programming models.
5. **Communication protocols**: Understand and explain communication protocols such as HTTP and TCP/IP.
6. **Basic database-driven application development**: Develop a basic database-driven application using a familiar application framework **(time permitting).**

1: Internet Fundamentals & HTML Basics

- Course introduction and overview
- How the internet works
- Introduction to client-server architecture
- Basic HTML structure and elements
- Creating your first webpage
- **Lab: Build a simple personal webpage using HTML**

2: HTML Deep Dive

- HTML5 semantic elements
- Forms and input types
- Tables and lists
- Media elements (images, audio, video)
- Best practices for HTML structure
- **Lab: Create a structured content page with various HTML elements**

3. Introduction to CSS

- CSS syntax and selectors
- Box model
- Colors, typography, and basic styling
- Linking CSS to HTML
- CSS specificity
- Lab: Style the previously created HTML pages

4. CSS Layout & Responsiveness

- Display properties
- Positioning
- Flexbox
- Media queries and responsive design
- CSS Grid introduction
- **Lab: Make your website responsive across different screen sizes**

5. JavaScript Fundamentals

- Introduction to JavaScript
- Variables, data types, and operators
- Control structures
- Functions
- Basic DOM manipulation
- **Lab: Add simple interactivity to your website**

6. JavaScript & DOM

- Events and event handling
- More DOM manipulation
- Forms validation
- Local storage
- JSON basics
- **Lab: Create an interactive form with validation**

7. Internet Protocols & Web Security

- HTTP/HTTPS protocols
- TCP/IP overview
- Web security basics
- CORS and same-origin policy
- Common security threats

8. Introduction to Backend Development

- Server-side concepts
- Introduction to APIs
- RESTful services
- HTTP methods (GET, POST, PUT, DELETE)

9. Databases & Storage

- Database concepts
- SQL vs NoSQL
- Basic database operations
- Connecting to a database

What is Web Technology?

Web technology is the collection of tools, languages, and frameworks used to create and maintain websites and web applications. It allows users to interact with information on the internet.

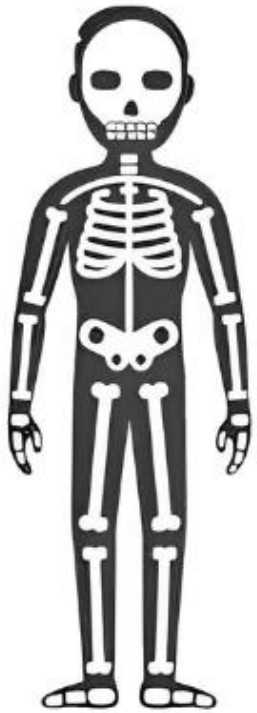
Components of web technology

- **Web browsers**: Software that displays web pages and retrieves information from the internet
- **Web servers**: Systems that store, process, and deliver web pages to users
- **Hypertext Markup Language (HTML)**: The standard language for creating web pages
- **Cascading Style Sheets (CSS)**: A language that describes how HTML documents are presented
- **JavaScript**: A programming language that creates interactive effects and dynamic content
- **Databases**: Systems that store, retrieve, and query data

HTML



HTML the Skeleton



CSS



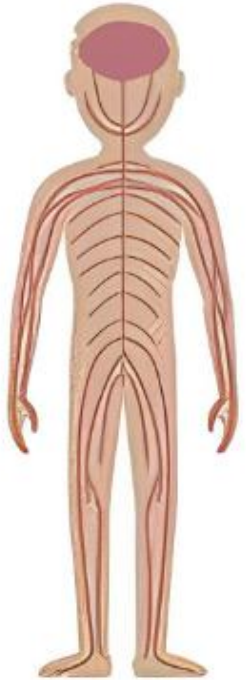
CSS the Skin

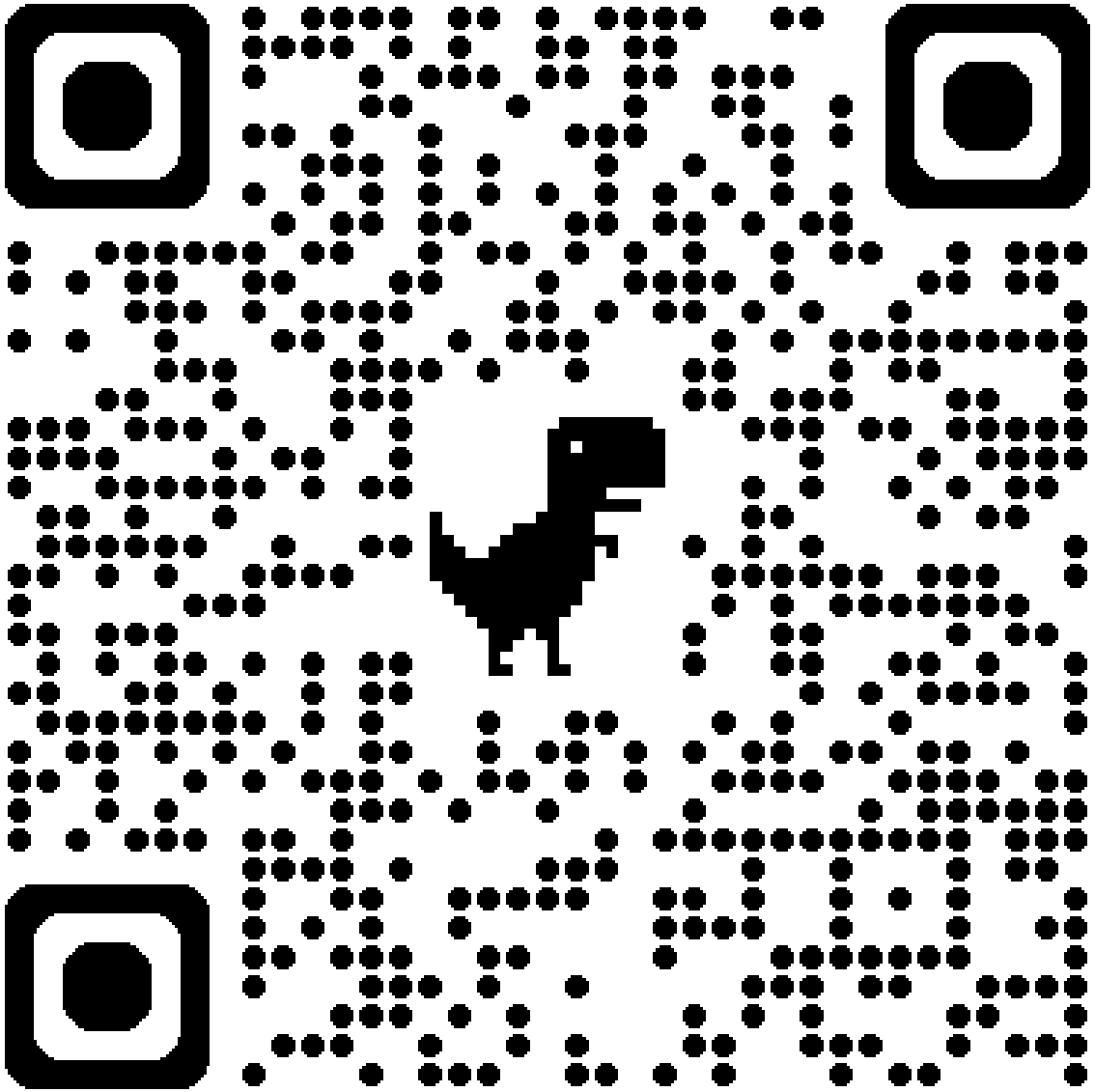


JS



Javascript the Brain





Web Essentials

CLIENTS

A "client" refers to any software application that requests services from another application (usually a server). It can be a software application, a computer, or any other device that can connect to a network.

Clients initiate communication with servers, send requests, and receive responses. They typically process and display the information received from the server.

Examples:

- A desktop email application like Microsoft Outlook is a client that requests emails from a mail server.
- A video game console is a client that connects to game servers to play online games.
- A mobile app that fetches data from a server is a client.

WEB CLIENT

- A web client is a specific type of client that uses a web browser to interact with web servers. It's essentially a program that can display web pages and interact with web applications.
- **Function:** Web clients send HTTP requests to web servers, receive HTML, CSS, and JavaScript code, and render it to display web pages. They also handle user interactions with web pages, such as clicking links or submitting forms.
- **Examples:**
  - Popular web browsers like Google Chrome, Mozilla Firefox, Safari, and Microsoft Edge are all web clients.
  - Any application that displays web content or interacts with web services through a browser is considered a web client.

KEY DIFFERENCES

Feature	Client	Web Client
Type	General term for any program or device requesting services	Specific type of client using a web browser
Communication	Can use various protocols (e.g., TCP/IP, SMTP)	Primarily uses HTTP or HTTPS
Interface	Can have a dedicated user interface or run in the background	Relies on a web browser for user interaction
Installation	May require installation on the user's device	Typically, does not require installation, accessed through a browser
Updates	Updates may need to be manually installed	Updates are usually handled automatically by the browser

SERVER

- A server is a computer or software program that provides services or resources to other computers or programs (called clients) over a network. It's a broad term that encompasses various types of servers, each with specific functions.
- **Function:** Servers store data, run applications, and manage network resources. They respond to requests from clients, providing them with the information or services they need.
- **Examples:**
  - A file server stores and manages files that can be accessed by multiple users.
  - A database server stores and manages data for applications.
  - A mail server handles email messages.
  - A game server hosts online games, allowing players to connect and play together.

WEB SERVER

- A web server is a specific type of server that serves web content to clients over the internet. It's essentially a computer that stores website files and delivers them to web browsers upon request.
- **Function:** Web servers receive HTTP requests from clients (web browsers), process those requests, and send back the corresponding web pages or resources (HTML, CSS, JavaScript, images, etc.).
- **Examples:**
  - Apache HTTP Server: A popular open-source web server.
  - Nginx: Another widely used open-source web server known for its performance.
  - Microsoft IIS: A web server developed by Microsoft for Windows environments.

KEY DIFFERENCES

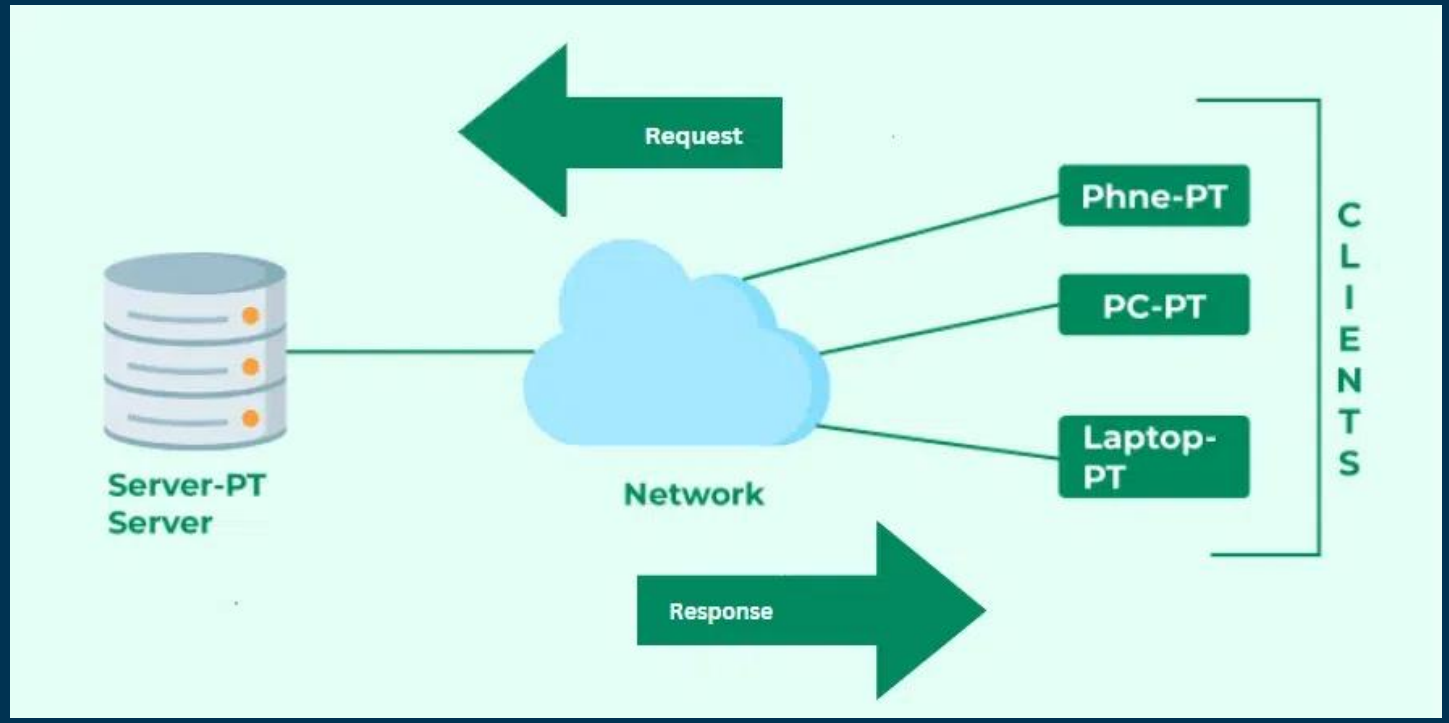
Feature	Server	Web Server
Type	General term for any computer or program providing services	Specific type of server serving web content
Communication	Can use various protocols (e.g., TCP/IP, SMTP, FTP)	Primarily uses HTTP or HTTPS
Content	Can serve various types of data (files, database records, emails)	Specifically serves web content (HTML, CSS, JavaScript, images)
Purpose	To provide a wide range of services to clients	To host websites and deliver web content to clients


StreetAddict

CLIENT SERVER MODEL

Client-server model is a distributed application structure that partitions tasks or workloads between the providers of a resource or service, called servers, and service requesters called clients.

In the client-server architecture, when the client computer sends a request for data to the server through the internet, the server accepts the requested process and delivers the data packets requested back to the client. Clients do not share any of their resources. Examples of the Client-Server Model are Email, World Wide Web, etc.



How the Browser Interacts With the Servers?

User enters the **URL**(Uniform Resource Locator) of the website or file. The Browser then requests the **DNS** (DOMAIN NAME SYSTEM) Server.

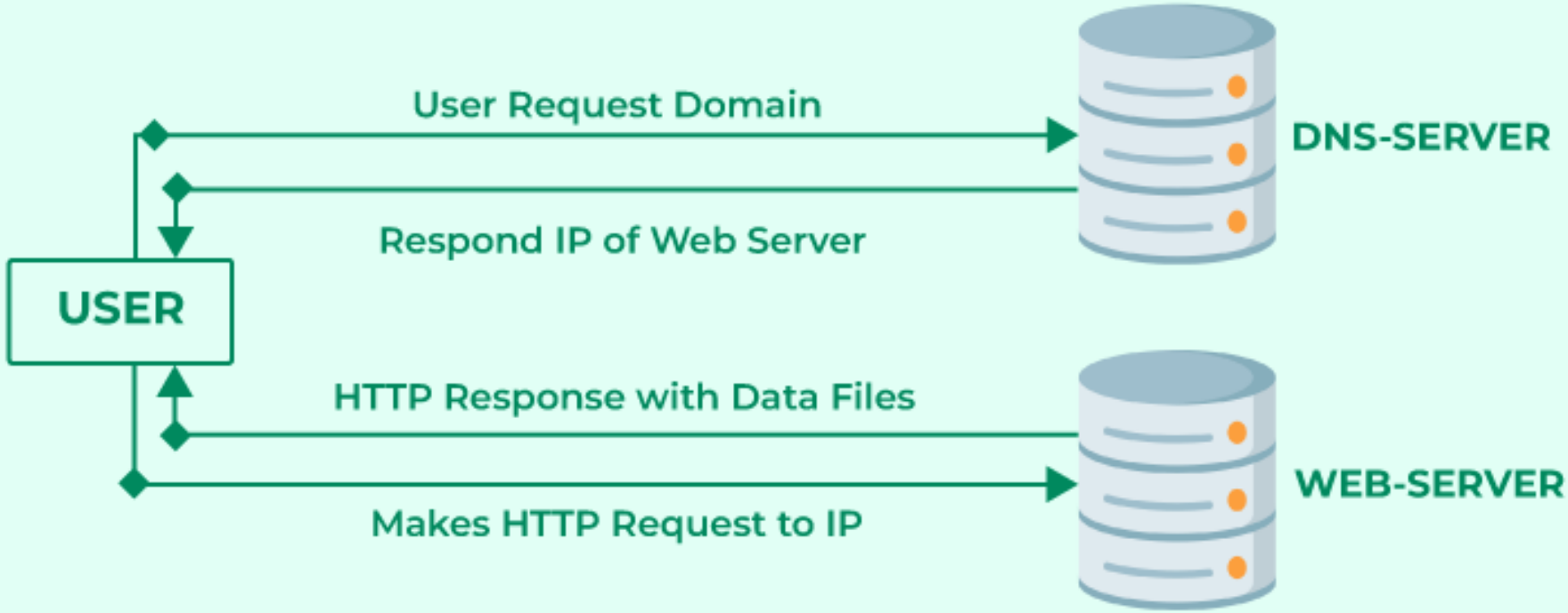
DNS Server lookup for the address of the **WEB Server**.

The **DNS Server** responds with the **IP address** of the **WEB Server**.

The Browser sends over an **HTTP/HTTPS** request to the **WEB Server's IP** (provided by the **DNS server**).

The Server sends over the necessary files for the website.

The Browser then renders the files and the website is displayed. This rendering is done with the help of **DOM** (Document Object Model) interpreter, **CSS** interpreter, and **JS Engine** collectively known as the **JIT** or (Just in Time) Compilers.


User Request Domain
DNS-SERVER
Respond IP of Web Server
USER
HTTP Response with Data Files
WEB-SERVER
Makes HTTP Request to IP

The Internet and the WWW(World Wide Web)

What is the Internet?

The internet is a globally connected network system facilitating worldwide communication and access to data resources through a vast collection of private, public, business, academic and government networks. It is governed by agencies like the Internet Assigned Numbers Authority (or IANA) that establish universal protocols.

INTERconnected **NET**work

WWW(World Wide Web)

World Wide Web, which is also known as a Web, is a collection of websites or web pages stored in web servers and connected to local computers through the internet. These websites contain text pages, digital images, audios, videos, etc. Users can access the content of these sites from any part of the world over the internet using their devices such as computers, laptops, cell phones, etc. The WWW, along with internet, enables the retrieval and display of text and media to your device.

Evolution of the Internet

- The internet originated with the U.S. government, which began building a computer network in the 1969 known as ARPANET.

  ARPA = **Advanced Research Projects Agency** sponsored by the American

- Department of Defense (DOD). Designed to connect military research centers. Distributed computer system able to survive a nuclear attack. Four nodes: at UCLA (Los Angeles), Stanford Research Institute, UCSB (Santa Barbara), and University of Utah

Evolution of the Internet

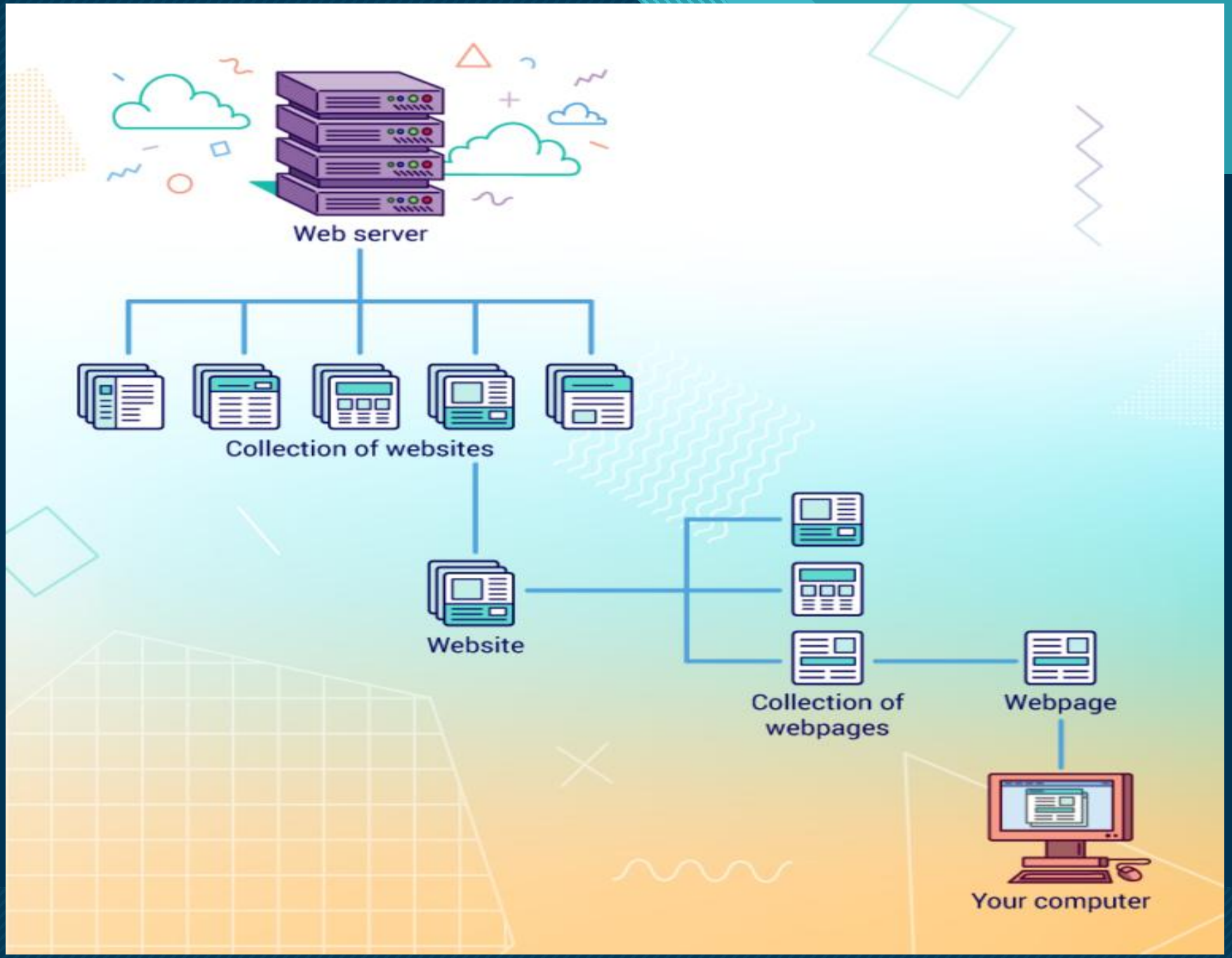
- Problem: ARPAnet could connect only networks of the same type.
- In 1970, ARPA starts developing the Transmission Control Protocol / Internet Protocol (TCP/IP), a technology for connecting networks of different types (produced by different companies).
- Other networks appear, such as CSNET and BIT
- The concept of **internetworking** was developed. A network connects through a gateway that facilitates communication with networks of different types.

Evolution of the Internet

- The name Internet starts to be used.
- In 1980 NSFnet is created by National Science Foundation to connect supercomputer centers for academic research.
- By 1990, NSFnet replaces ARPAnet, which ceases to exists.
- In 1985, the U.S. National Science Foundation (NSF) commissioned the development of a university network backbone called NSFNET.

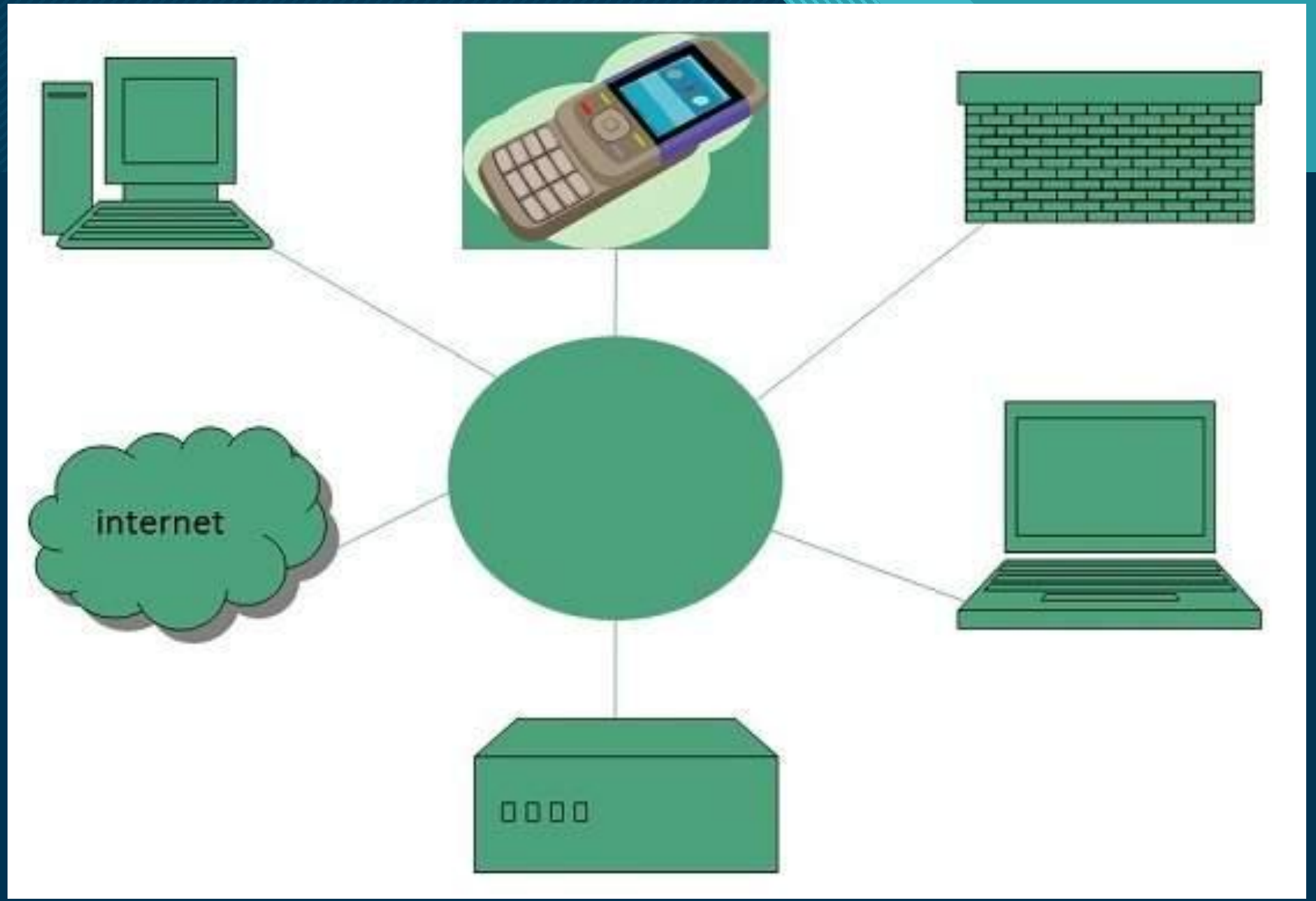
Evolution of the Web

- The World Wide Web was invented by a British scientist, Tim Berners-Lee in 1989.
- Originally, it was developed by him to fulfill the need of automated information sharing between scientists across the world, so that they could easily share the data and results of their experiments and studies with each other.
- The objective of the World Wide Web Consortium (W3C) is to develop guidelines that ensure long-term growth for the web.


Web server
Collection of websites
Website
Collection of webpages
Webpage
Your computer

Internet Overview

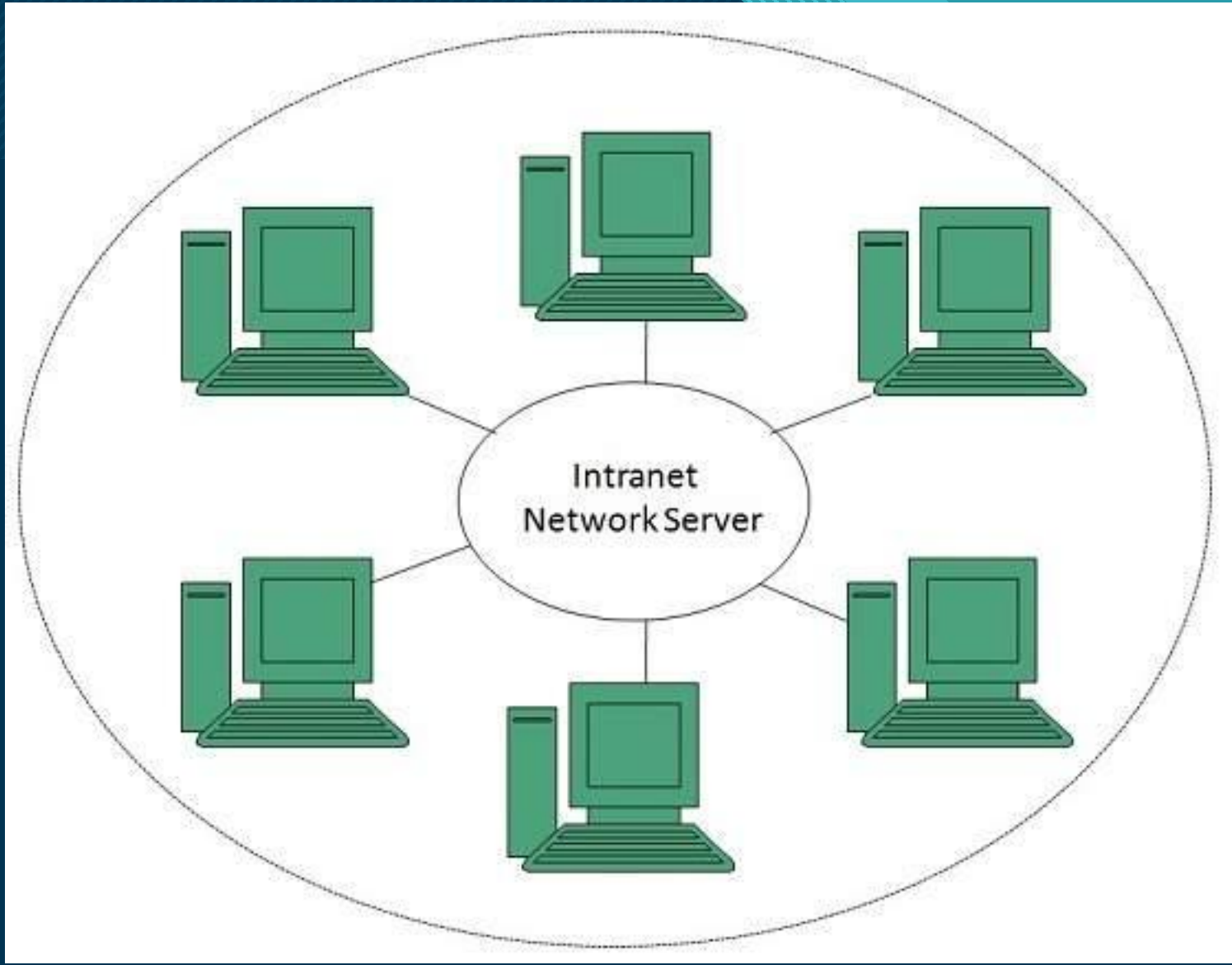
- Internet uses the standard Internet Protocol (TCP/IP).
- Every computer in internet is identified by a unique IP address.
- IP Address is a unique set of numbers (such as 110.22.33.114) which identifies a computer location.
- A special computer DNS (Domain Name Server) is used to give name to the IP Address so that user can locate a computer by a name.
- For example, a DNS server will resolve a name https://adelekeuniversity.edu.ng/ to a particular IP address to uniquely identify the computer on which this website is hosted.


internet

Intranet

Intranet is defined as private network of computers within an organization with its own server and firewall. Moreover we can define Intranet as:

- Intranet is system in which multiple PCs are networked to be connected to each other. PCs in intranet are not available to the world outside of the intranet.
- Usually each company or organization has their own Intranet network and members/employees of that company can access the computers in their intranet.
- Every computer in internet is identified by a unique IP address.
- Each computer in Intranet is also identified by a IP Address, which is unique among the computers in that Intranet.


Intranet
Network Server

Reference Model

Reference Model offers a means of standardization which is acceptable worldwide. Since people using the computer network are located over a wide physical range and their network devices might have heterogeneous architecture. In order to provide communication among heterogeneous devices, we need a standardized model i.e. a reference model, which would provide us way how these devices can communicate regardless their architecture.

We have two reference models such as OSI model and TCP/IP reference model, however, the OSI model is a hypothetical one but the TCP/IP is absolutely practical model.

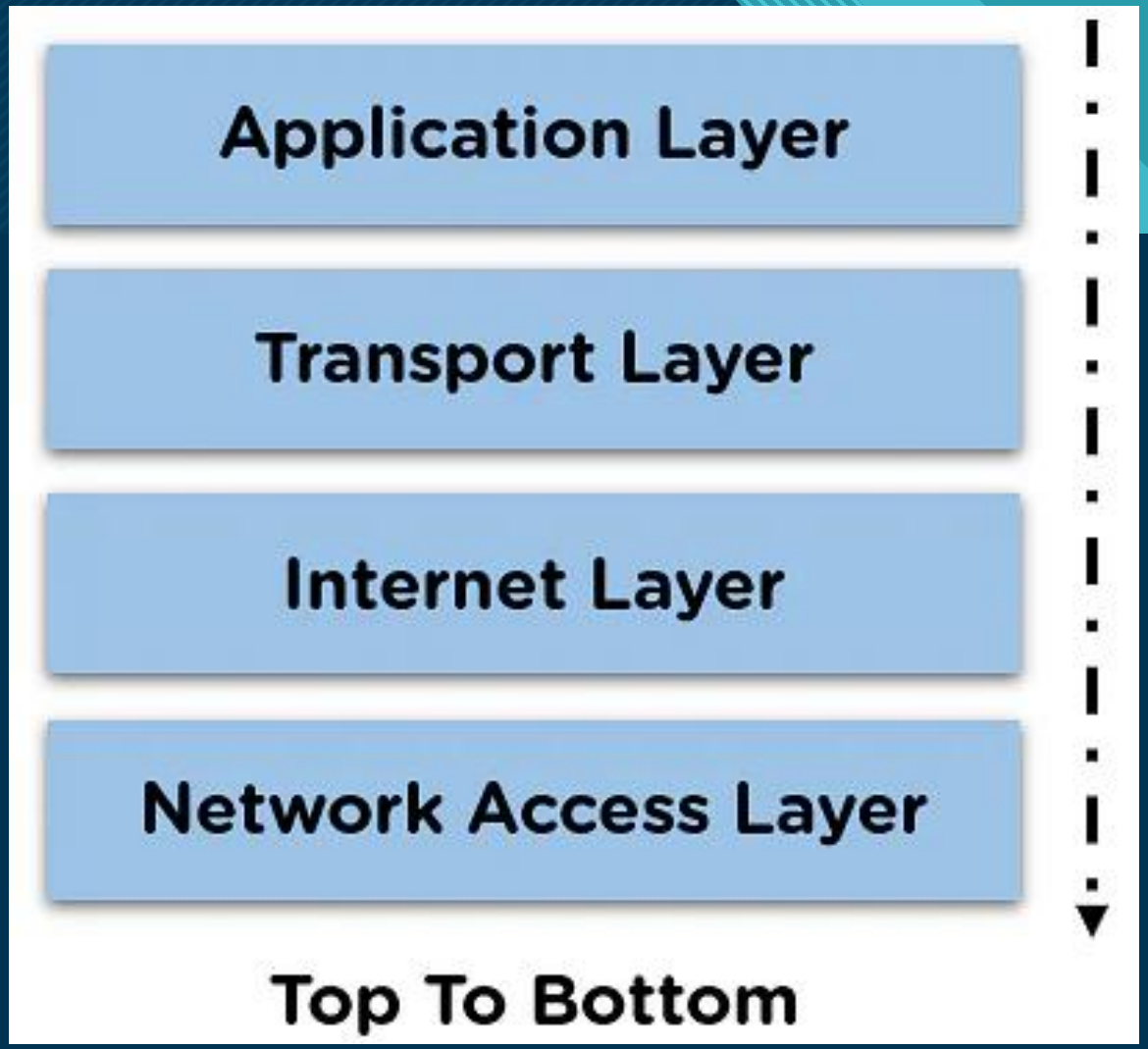
TCP/IP (Transmission Control Protocol/ Internet Protocol)

These are a set of standard rules that allows different types of computers to communicate with each other. The IP protocol ensures that each computer that is connected to the Internet is having a specific serial number called the IP address. TCP specifies how data is exchanged over the internet and how it should be broken into IP packets. It also makes sure that the packets have information about the source of the message data, the destination of the message data, the sequence in which the message data should be re-assembled, and checks if the message has been sent correctly to the specific destination. The TCP is also known as a connection-oriented protocol.

The TCP/IP model is divided into four different layers:

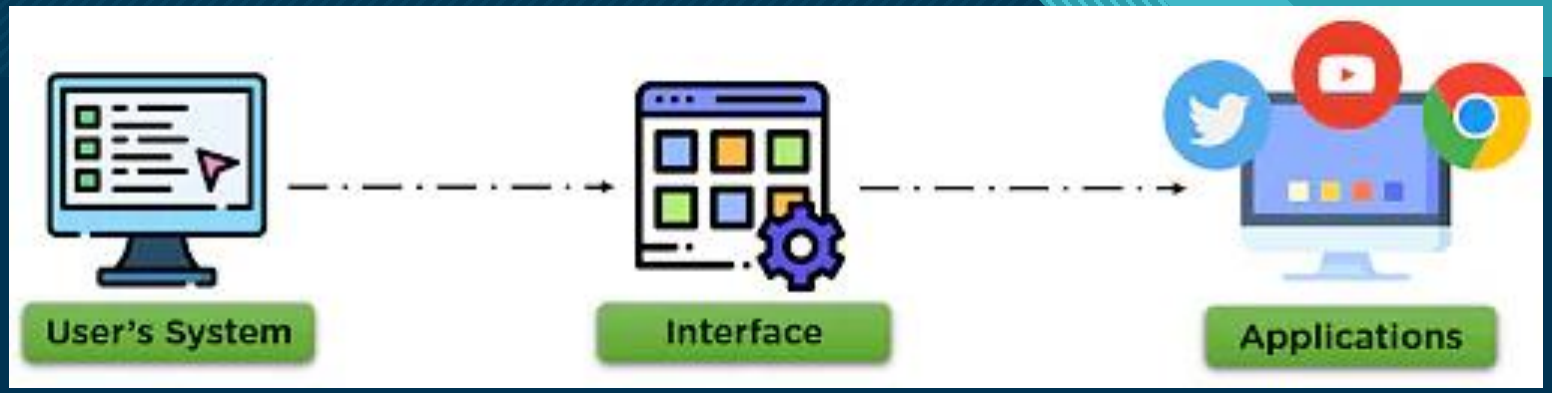
- Application layer
- Transport layer
- Internet layer
- Network Access layer

Each layer performs a specific task on the data that is being transmitted over the network channel, and data moves from one layer to another in a preset pattern



The above model represents the flow of data when it is being transmitted from the sender side. In the case of data being received, the layers of the model work in reverse order.

Application Layer



This is the topmost layer which indicates the applications and programs that utilize the TCP/IP model for communicating with the user through applications and various tasks performed by the layer, including data representation for the applications executed by the user and forwards it to the transport layer.

The application layer maintains a smooth connection between the application and user for data exchange and offers various features as remote handling of the system, e-mail services, etc.

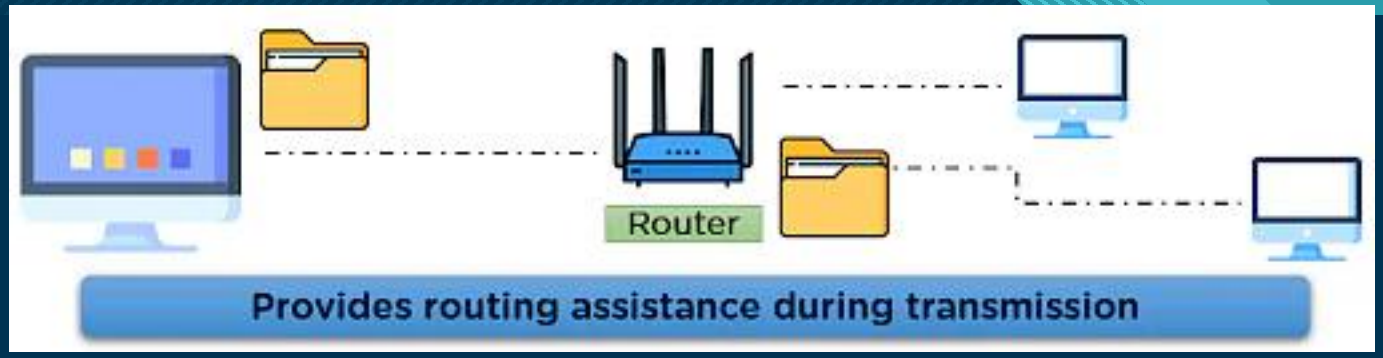
Transport Layer



This layer is responsible for establishing the connection between the sender and the receiver device and also performs the task of dividing the data from the application layer into packets, which are then used to create sequences.

It also performs the task of maintaining the data, i.e., to be transmitted without error, and controls the data flow rate over the communication channel for smooth transmission of data.

Internet Layer

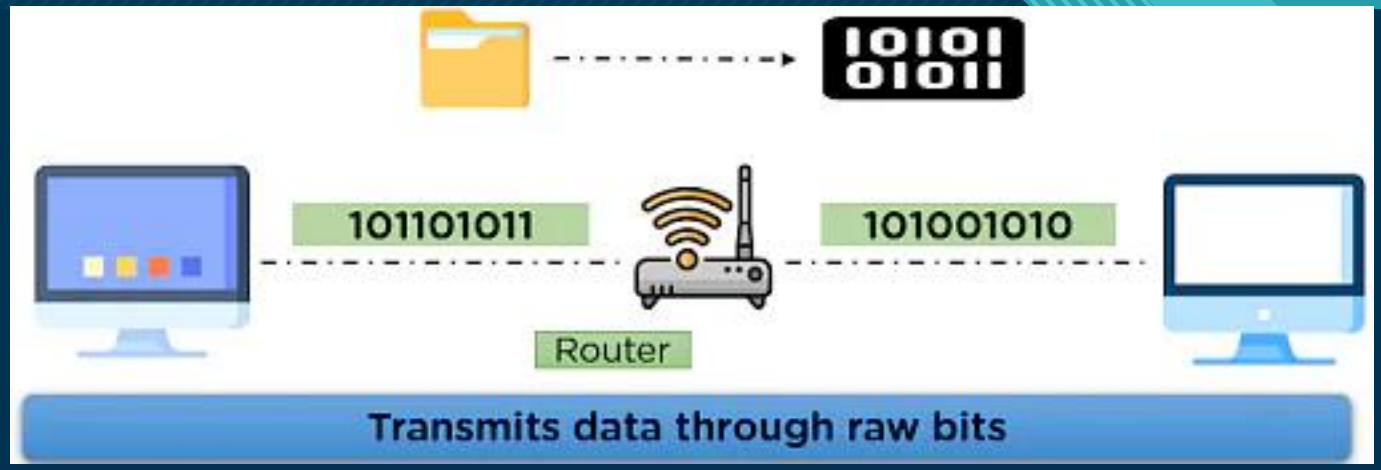


The Internet layer performs the task of controlling the transmission of the data over the network modes and enacts protocols related to the various steps related to the transmission of data over the channel, which is in the form of packets sent by the previous layer.

This layer performs many important functions in the TCP/IP model, some of which are:

1. It is responsible for specifying the path that the data packets will use for transmission.
2. This layer is responsible for providing IP addresses to the system for the identification matters over the network channel.

Network Access Layer



This layer is the combination of data-link and physical layer, where it is responsible for maintaining the task of sending and receiving data in raw bits, i.e., in binary format over the physical communication modes in the network channel.

- It uses the physical address of the system for mapping the path of transmission over the network channel.

Most Common TCP/IP Protocols

- **TCP:** Transmission Control Protocol is an internet protocol suite which breaks up the message into TCP Segments and reassembling them at the receiving side.
- **IP:** An Internet Protocol address that is also known as an IP address is a numerical label. It is assigned to each device that is connected to a computer network which uses the IP for communication. Its routing function allows internetworking and essentially establishes the Internet. Combination of IP with a TCP allows developing a virtual connection between a destination and a source.
- **HTTP:** The Hypertext Transfer Protocol is a foundation of the World Wide Web. It is used for transferring web pages and other such resources from the HTTP server or web server to the web client or the HTTP client. Whenever you use a web browser like Google Chrome or Firefox, you are using a web client. It helps HTTP to transfer web pages that you request from the remote servers.

- **HTTPS(HyperText Transfer Protocol Secure):** HTTPS is an extension of the Hypertext Transfer Protocol (HTTP). It is used for secure communication over a computer network with the SSL/TLS protocol for encryption and authentication. So, generally, a website has an HTTP protocol but if the website is such that it receives some sensitive information such as credit card details, debit card details, OTP, etc then it requires an SSL certificate installed to make the website more secure. So, before entering any sensitive information on a website, we should check if the link is HTTPS or not. If it is not HTTPS then it may not be secure enough to enter sensitive information.
- **SMTP:** SMTP stands for Simple mail transfer protocol. This protocol supports the e-mail is known as a simple mail transfer protocol. This protocol helps you to send the data to another email address.
- **SNMP:** SNMP stands for Simple Network Management Protocol. It is a framework which is used for managing the devices on the internet by using the TCP/IP protocol.
- **DNS**: DNS stands for Domain Name System. An IP address that is used to identify the connection of a host to the internet uniquely. However, users prefer to use names instead of addresses for that DNS.

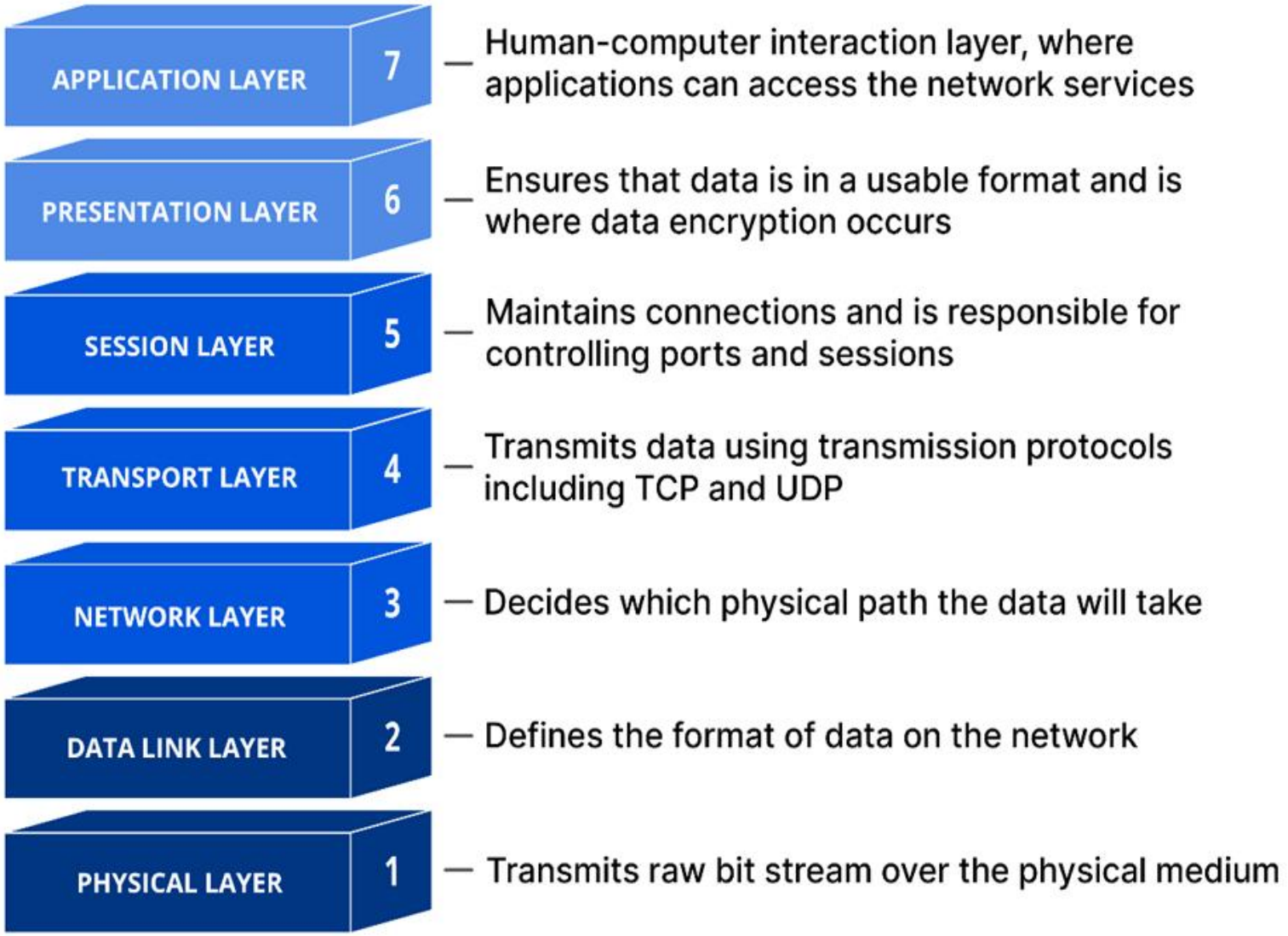
- **TELNET:** TELNET stands for Terminal Network. It establishes the connection between the local and remote computer. It established connection in such a manner that you can simulate your local system at the remote system.
- **FTP:** FTP stands for File Transfer Protocol. It is a mostly used standard protocol for transmitting the files from one machine to another.
- **SFTP(Secure File Transfer Protocol):** SFTP which is also known as SSH FTP refers to File Transfer Protocol (FTP) over Secure Shell (SSH) as it encrypts both commands and data while in transmission. SFTP acts as an extension to SSH and encrypts files and data then sends them over a secure shell data stream. This protocol is used to remotely connect to other systems while executing commands from the command line.
- **POP3(Post Office Protocol 3):** POP3 stands for Post Office Protocol version 3. It has two Message Access Agents (MAAs) where one is client MAA (Message Access Agent) and another is server MAA(Message Access Agent) for accessing the messages from the mailbox. This protocol helps us to retrieve and manage emails from the mailbox on the receiver mail server to the receiver's computer. This is implied between the receiver and receiver mail server. It can also be called as one way client server protocol. The POP3 WORKS ON THE 2 PORTS I.E. PORT 110 AND PORT 995.

- **PPP**(Point to Point Protocol): It is a communication protocol that is used to create a direct connection between two communicating devices. This protocol defines the rules using which two devices will authenticate with each other and exchange information with each other. For example, A user connects his PC to the server of an Internet Service Provider also uses PPP. Similarly, for connecting two routers for direct communication it uses PPP.
- **UDP**: User Datagram Protocol is responsible for identifying errors, and other tasks during the transmission of information. UDP maintains various fields for data transmission such as:
    - Source Port Address: This port is responsible for designing the application that makes up the message to be transmitted.
    - Destination Port Address: This port receives the message sent from the sender side.
    - Total Length: The total number of bytes of the user datagram.
    - Checksum: Used for error detection of the message at the destination side.
- **ARP**: This protocol refers to the Address Resolution Protocol that is responsible for finding the physical address using the IP address.

OSI Model

OSI is acronym of Open System Interface. This model is developed by the International organization of Standardization (ISO) and therefore also referred as ISO-OSI Model.

The OSI model consists of seven layers. Each layer has a specific function, however each layer provide services to the layer above.


APPLICATION LAYER
7
Human-computer interaction layer, where applications can access the network services
PRESENTATION LAYER
6
Ensures that data is in a usable format and is where data encryption occurs
SESSION LAYER
5
Maintains connections and is responsible for controlling ports and sessions
TRANSPORT LAYER
4
Transmits data using transmission protocols including TCP and UDP
NETWORK LAYER
3
Decides which physical path the data will take
DATA LINK LAYER
2
Defines the format of data on the network
PHYSICAL LAYER
1
Transmits raw bit stream over the physical medium

All People Seem To Need Data Processing

Application Layer

Presentation Layer

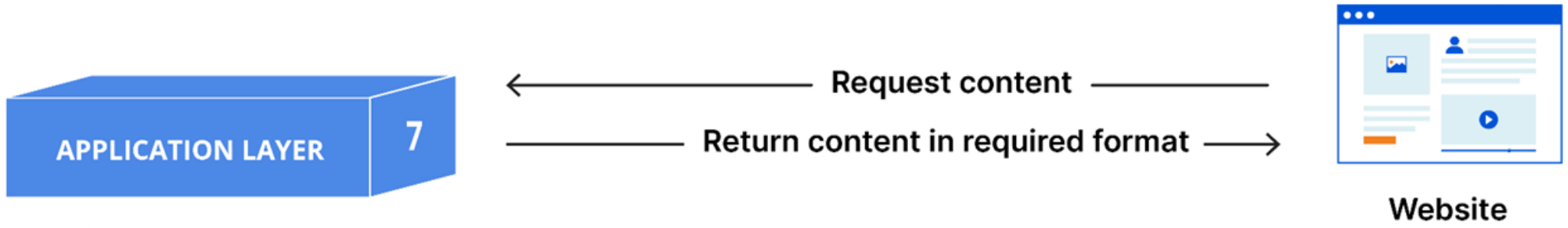
Session Layer

Transport Layer

Network Layer

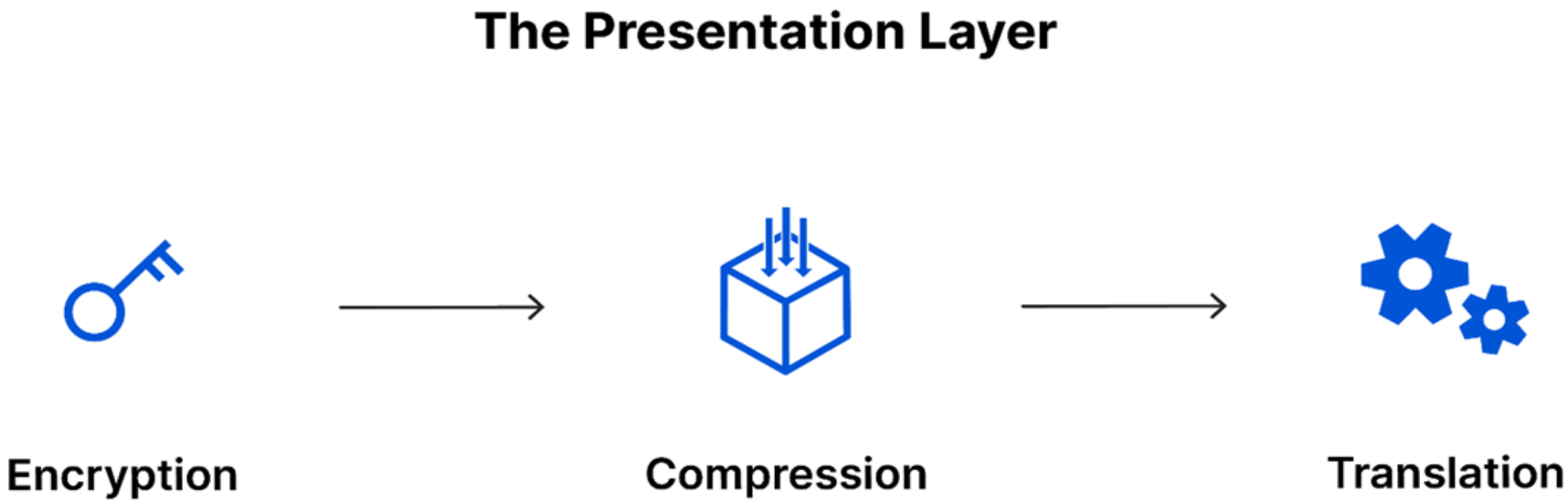
Data Link Layer

Physical Layer



This is the only layer that directly interacts with data from the user. Software applications like web browsers and email clients rely on the application layer to initiate communications. But it should be made clear that client software applications are not part of the application layer; rather the application layer is responsible for the protocols and data manipulation that the software relies on to present meaningful data to the user.

06. THE PRESENTATION LAYER



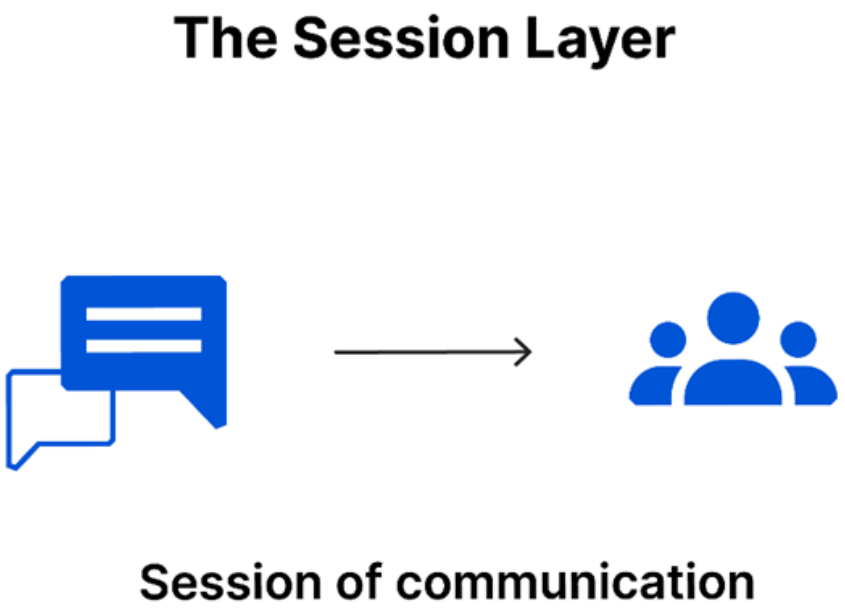
This layer is primarily responsible for preparing data so that it can be used by the application layer; in other words, layer 6 makes the data presentable for applications to consume. The presentation layer is responsible for translation, encryption, and compression of data.

Two communicating devices communicating may be using different encoding methods, so layer 6 is responsible for translating incoming data into a syntax that the application layer of the receiving device can understand.

If the devices are communicating over an encrypted connection, layer 6 is responsible for adding the encryption on the sender’s end as well as decoding the encryption on the receiver's end so that it can present the application layer with unencrypted, readable data.

Finally the presentation layer is also responsible for compressing data it receives from the application layer before delivering it to layer 5. This helps improve the speed and efficiency of communication by minimizing the amount of data that will be transferred.

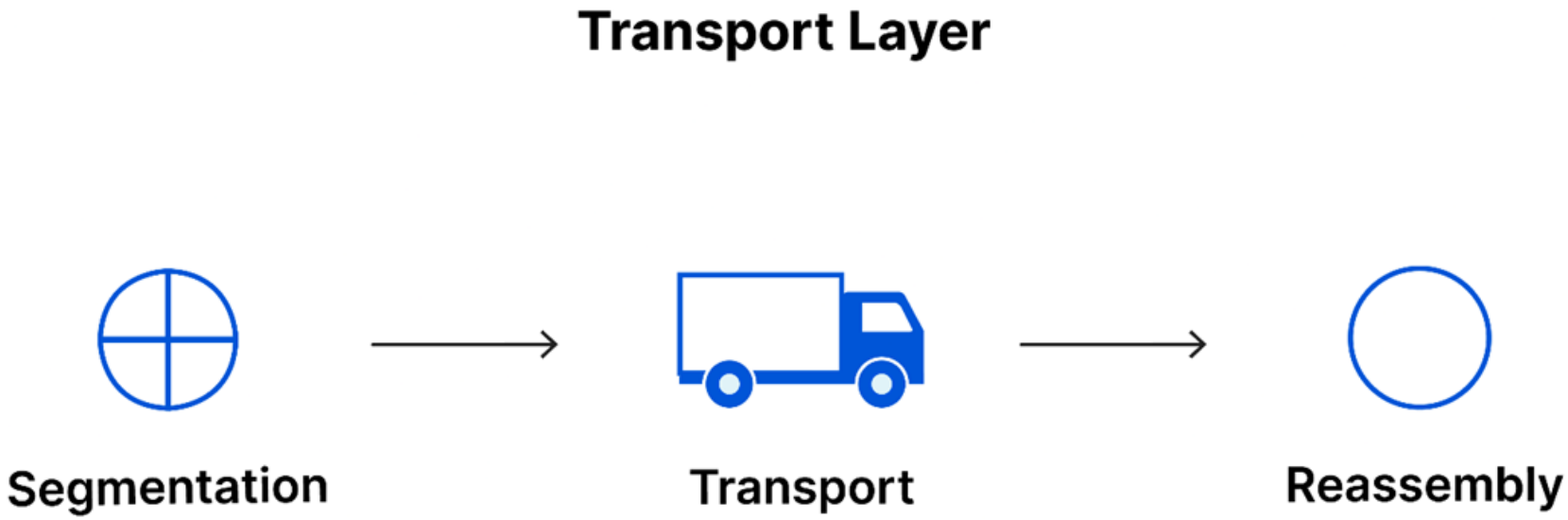
05. THE SESSION LAYER



This is the layer responsible for opening and closing communication between the two devices. The time between when the communication is opened and closed is known as the session. The session layer ensures that the session stays open long enough to transfer all the data being exchanged, and then promptly closes the session in order to avoid wasting resources.

The session layer also synchronizes data transfer with checkpoints. For example, if a 100 megabyte file is being transferred, the session layer could set a checkpoint every 5 megabytes. In the case of a disconnect or a crash after 52 megabytes have been transferred, the session could be resumed from the last checkpoint, meaning only 50 more megabytes of data need to be transferred. Without the checkpoints, the entire transfer would have to begin again from scratch.

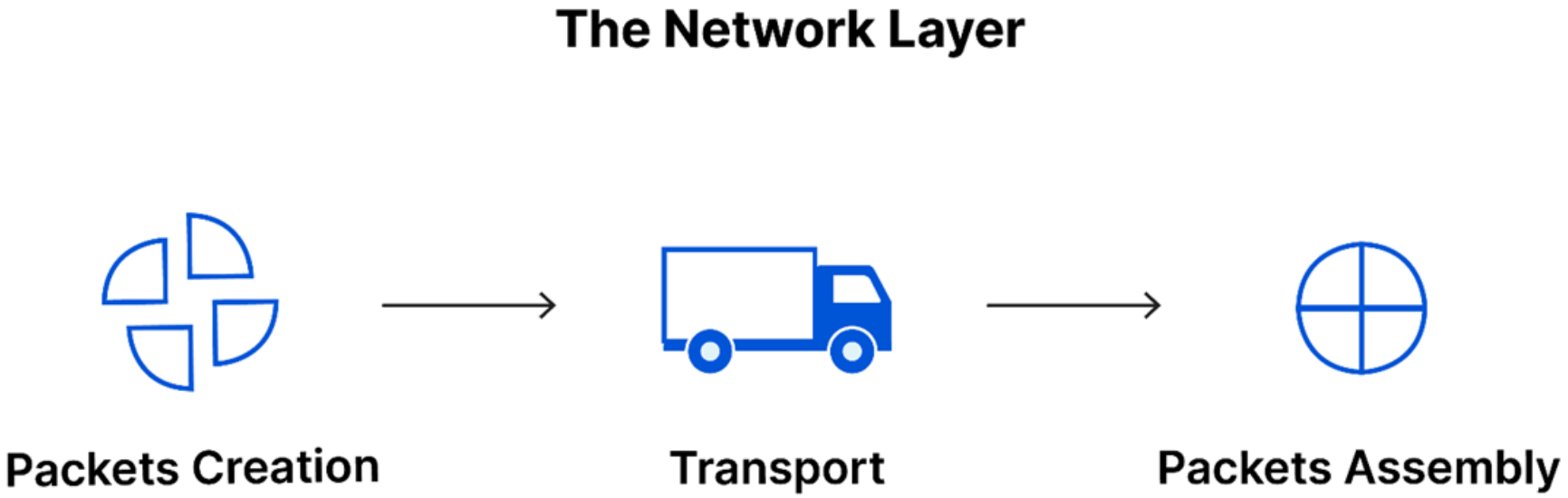
04. THE TRANSPORT LAYER



Layer 4 is responsible for end-to-end communication between the two devices. This includes taking data from the session layer and breaking it up into chunks called segments before sending it to layer 3. The transport layer on the receiving device is responsible for reassembling the segments into data the session layer can consume.

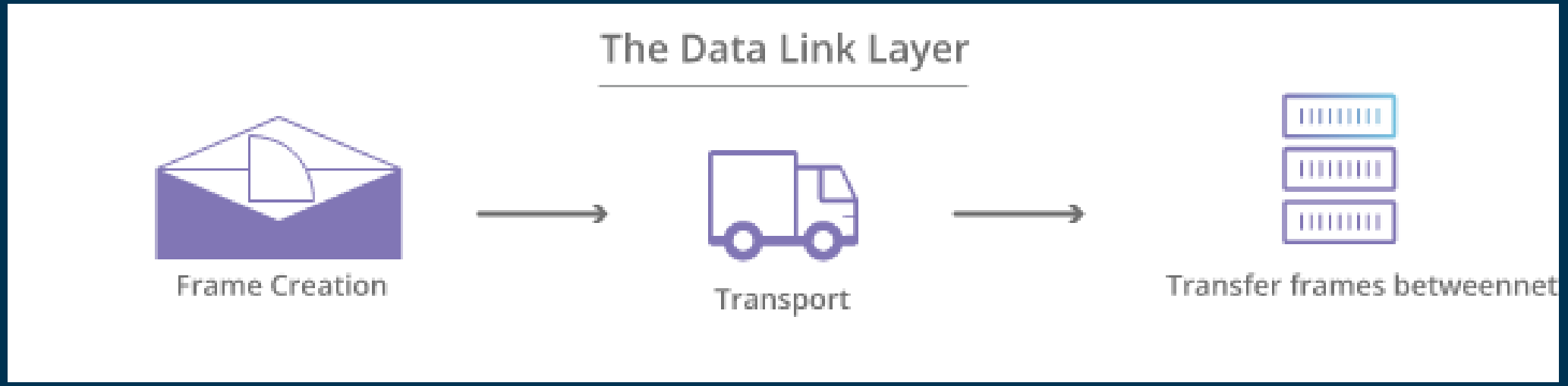
The transport layer is also responsible for flow control and error control. Flow control determines an optimal speed of transmission to ensure that a sender with a fast connection does not overwhelm a receiver with a slow connection. The transport layer performs error control on the receiving end by ensuring that the data received is complete, and requesting a retransmission if it isn’t.

03. THE NETWORK LAYER



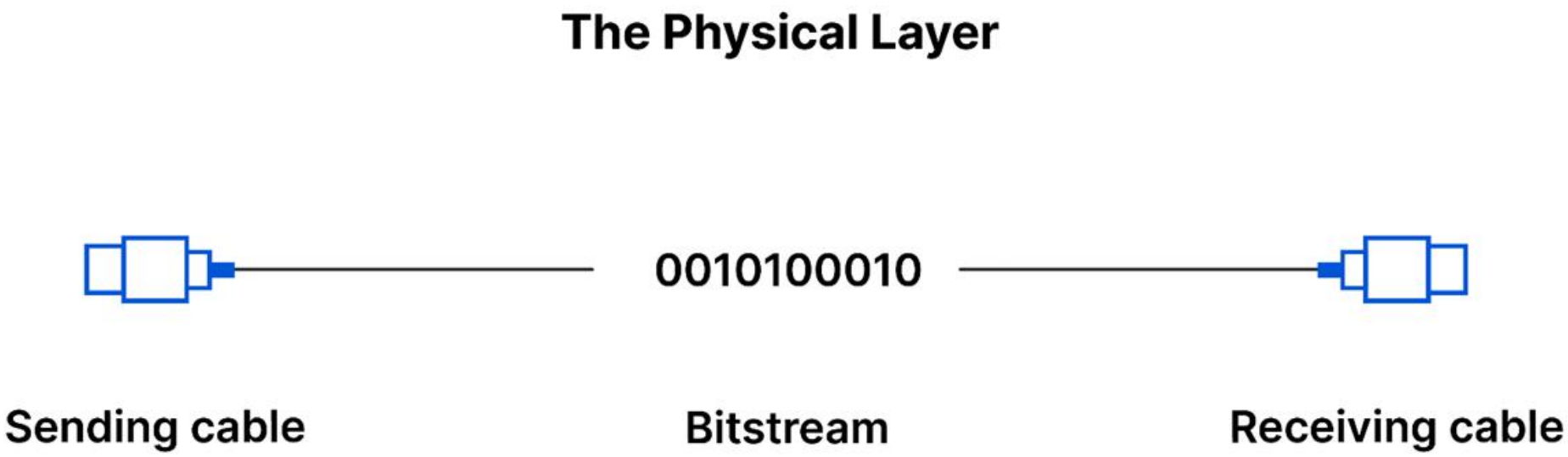
The network layer is responsible for facilitating data transfer between two different networks. If the two devices communicating are on the same network, then the network layer is unnecessary. The network layer breaks up segments from the transport layer into smaller units, called packets, on the sender’s device, and reassembling these packets on the receiving device. The network layer also finds the best physical path for the data to reach its destination; this is known as routing.

02. THE DATA LINK LAYER



The data link layer is very similar to the network layer, except the data link layer facilitates data transfer between two devices on the *same* network. The data link layer takes packets from the network layer and breaks them into smaller pieces called frames. Like the network layer, the data link layer is also responsible for flow control and error control in intra-network communication (The transport layer only does flow control and error control for inter-network communications).

01. THE PHYSICAL LAYER



This layer includes the physical equipment involved in the data transfer, such as the cables and switches. This is also the layer where the data gets converted into a bit stream, which is a string of 1s and 0s. The physical layer of both devices must also agree on a signal convention so that the 1s can be distinguished from the 0s on both devices.

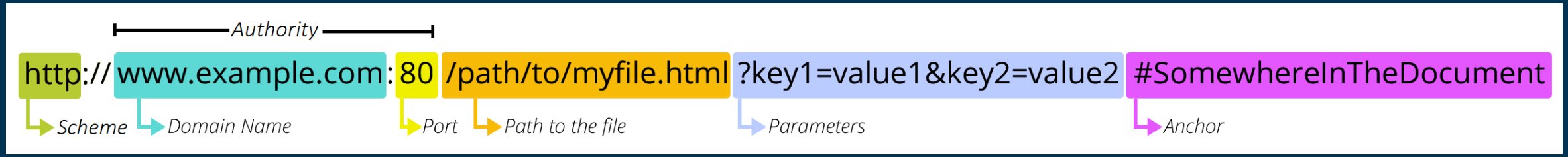
TCP/IP	OSI Model
TCP/IP model depends on standard protocols about which the computer network has created. It is a connection protocol that assigns the network of hosts over the internet.	OSI is a generic, protocol independent standard. It is acting as an interaction gateway between the network and the final-user.
It uses a horizontal approach.	It uses a vertical approach.
In 1982, the TCP/IP model became the standard language of ARPANET.	In 1984, the OSI model was introduced by the International Organisation of Standardization (ISO).
It is a communication protocol that is based on standard protocols and allows the connection of hosts over a network.	It is a structured model which deals which the functioning of a network.

URL (Uniform Resource Locator)

- A **URL** (Uniform Resource Locator) is the address of a unique resource on the internet. It is one of the key mechanisms used by browsers to retrieve published resources, such as HTML pages, CSS documents, images, and so on.
- Here are some examples of URLs:
- `https://developer.mozilla.org`
- `https://developer.mozilla.org/en-US/docs/Learn_web_development/`
- `https://developer.mozilla.org/en-US/search?q=URL`

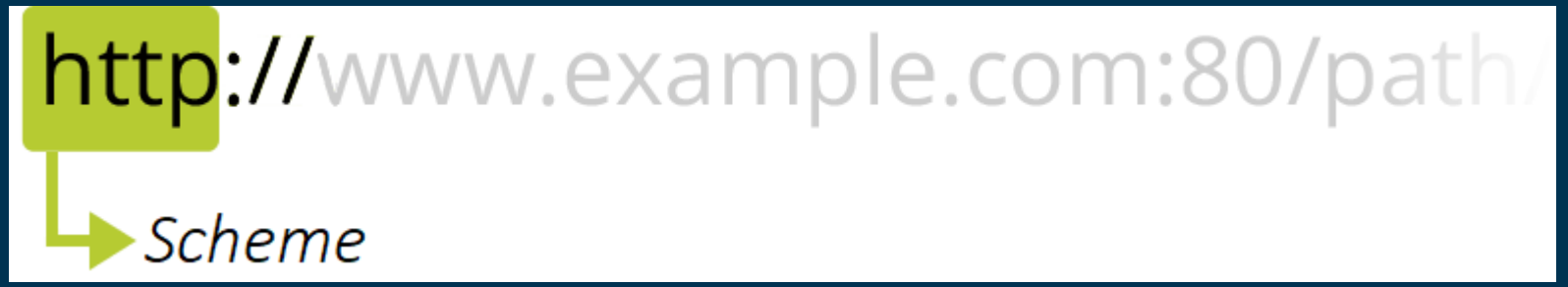
URL (Uniform Resource Locator)

- A URL is composed of different parts, some mandatory and others optional. The most important parts are highlighted on the URL below (details are provided in the following sections):



URL (Uniform Resource Locator)

- **Scheme**



- The first part of the URL is the *scheme*, which indicates the protocol that the browser must use to request the resource (a protocol is a set method for exchanging or transferring data around a computer network). Usually for websites the protocol is HTTPS or HTTP (its unsecured version).

URL (Uniform Resource Locator)

- **Authority**



Next follows the authority, which is separated from the scheme by the character pattern `://`. If present the authority includes both the domain (e.g. `www.example.com`) and the port (80), separated by a colon:

- The domain indicates which Web server is being requested. Usually this is a domain name, but an IP address may also be used (but this is rare as it is much less convenient).
- The port indicates the technical "gate" used to access the resources on the web server. It is usually omitted if the web server uses the standard ports of the HTTP protocol (80 for HTTP and 443 for HTTPS) to grant access to its resources. Otherwise, it is mandatory.

URL (Uniform Resource Locator)

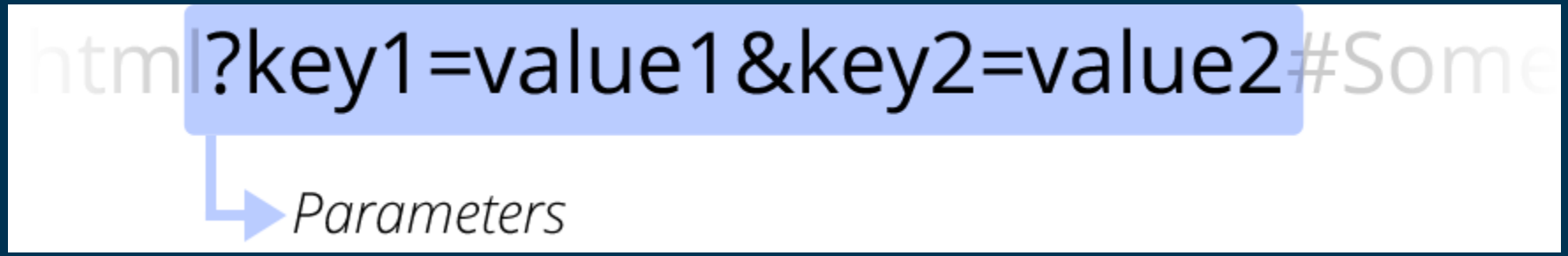
- **Path to resource**



- **`/path/to/myfile.html` is the path to the resource on the Web server. In the early days of the Web, a path like this represented a physical file location on the Web server. Nowadays, it is mostly an abstraction handled by Web servers without any physical reality.**

URL (Uniform Resource Locator)

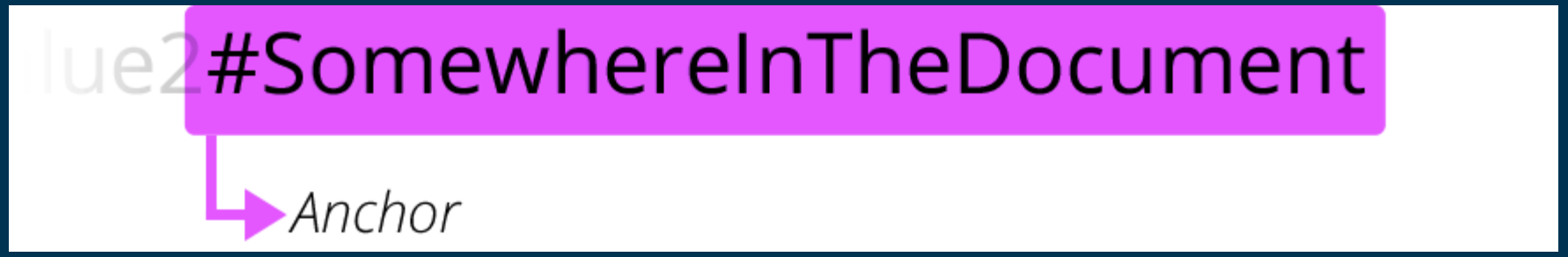
- **Parameters**



- `?key1=value1&key2=value2` are extra parameters provided to the Web server. Those parameters are a list of key/value pairs separated with the & symbol. The Web server can use those parameters to do extra stuff before returning the resource. Each Web server has its own rules regarding parameters, and the only reliable way to know if a specific Web server is handling parameters is by asking the Web server owner.

URL (Uniform Resource Locator)

- **Anchor**



- `#SomewhereInTheDocument` is an anchor to another part of the resource itself. An anchor represents a sort of "bookmark" inside the resource, giving the browser the directions to show the content located at that "bookmarked" spot. On an HTML document, for example, the browser will scroll to the point where the anchor is defined; on a video or audio document, the browser will try to go to the time the anchor represents. It is worth noting that the part after the #, also known as the fragment identifier, is never sent to the server with the request.

DNS (Domain Name System)

- **What is DNS?**
- The Domain Name System (DNS) is the phonebook of the Internet. Humans access information online through domain names, like nytimes.com or espn.com. Web browsers interact through Internet Protocol (IP) addresses. DNS translates domain names to IP addresses so browsers can load Internet resources.
- Each device connected to the Internet has a unique IP address which other machines use to find the device. DNS servers eliminate the need for humans to memorize IP addresses such as **192.168.1.1** (in IPv4), or more complex newer alphanumeric IP addresses such as **2400:cb00:2048:1::c629:d7a2** (in IPv6).

DNS (Domain Name System)

- **How does DNS work?**
- The process of DNS resolution involves converting a hostname (such as www.example.com) into a computer-friendly IP address (such as 192.168.1.1). An IP address is given to each device on the Internet, and that address is necessary to find the appropriate Internet device - like a street address is used to find a particular home. When a user wants to load a webpage, a translation must occur between what a user types into their web browser (example.com) and the machine-friendly address necessary to locate the example.com webpage.
- In order to understand the process behind the DNS resolution, it’s important to learn about the different hardware components a DNS query must pass between. For the web browser, the DNS lookup occurs "behind the scenes" and requires no interaction from the user’s computer apart from the initial request.

DNS (Domain Name System)

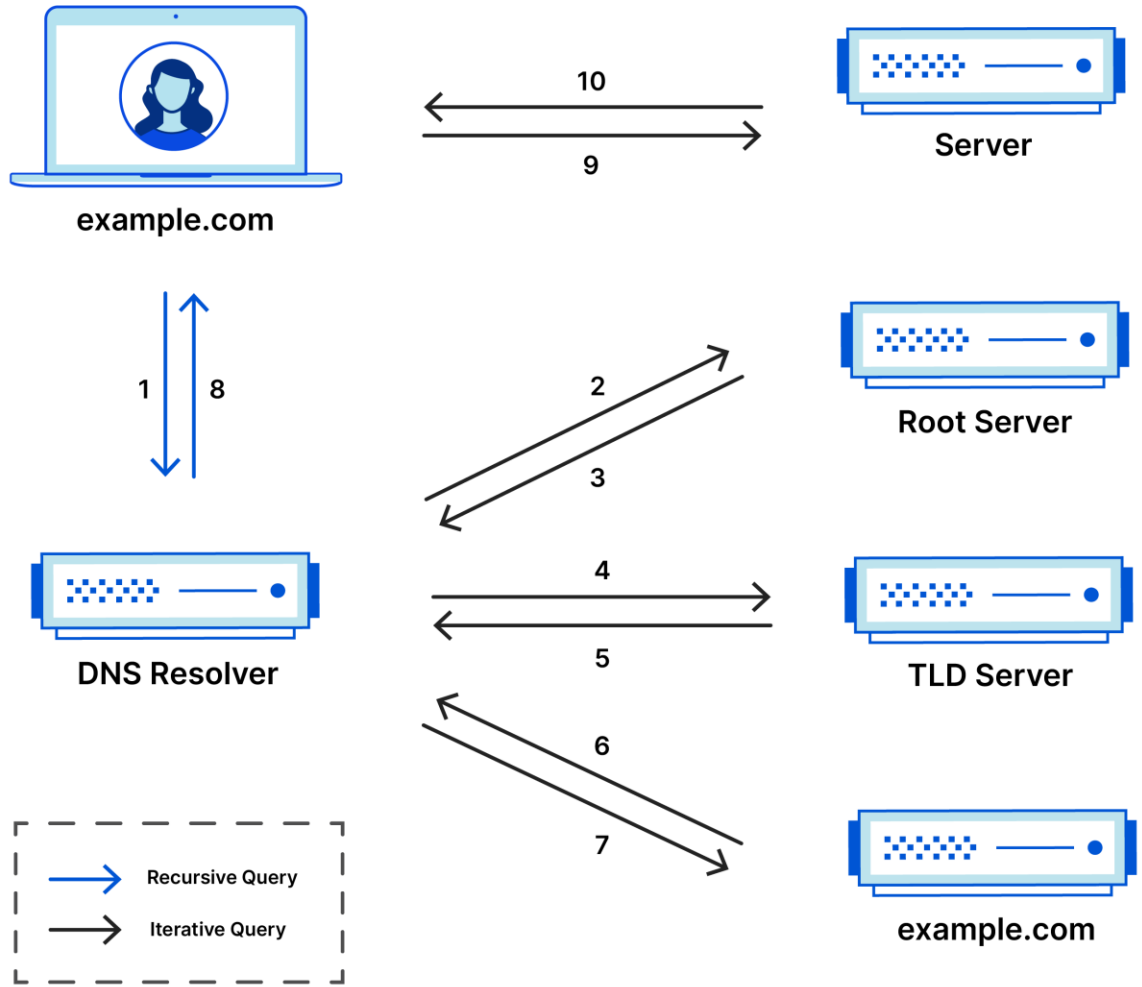
- **There are 4 DNS servers involved in loading a webpage:**
- DNS recursor - The recursor can be thought of as a librarian who is asked to go find a particular book somewhere in a library. The DNS recursor is a server designed to receive queries from client machines through applications such as web browsers. Typically, the recursor is then responsible for making additional requests in order to satisfy the client's DNS query.
- **Root nameserver** - The root server is the first step in translating (resolving) human readable host names into IP addresses. It can be thought of like an index in a library that points to different racks of books - typically it serves as a reference to other more specific locations.

DNS (Domain Name System)

- TLD nameserver - The top-level domain server (TLD) can be thought of as a specific rack of books in a library. This nameserver is the next step in the search for a specific IP address, and it hosts the last portion of a hostname (In example.com, the TLD server is “com”).

- Authoritative nameserver - This final nameserver can be thought of as a dictionary on a rack of books, in which a specific name can be translated into its definition. The authoritative nameserver is the last stop in the nameserver query. If the authoritative name server has access to the requested record, it will return the IP address for the requested hostname back to the DNS Recursor (the librarian) that made the initial request.

Complete DNS Lookup and Webpage Query



Structure of a domain name

A domain name can be divided into three main elements based on its structure—the top, second, and third level or subdomain.

- **Top-level domain**
- A top-level domain (TLD) is the final part of a web address. For example, the TLD for **`google.com`** is **`.com`**.
- Depending on the use cases and registration requirements, there are three types of TLDs.

Structure of a domain name

- **Generic top-level domain (gTLD)**
- As the name suggests, generic top-level domains are used for general purposes. So, anyone can register for these extensions without any restrictions.
- You might be familiar with some of the most popular generic TLDs:
- **.com** – initially created for commercial sites, it's now the most popular TLD, with almost half of all websites using it.
- **.net** – a great alternative to **.com**, this TLD is suitable for businesses in any sector.
- **.org** – unlike the other two, **.org** is more geared toward non-profit organizations. It's the second most popular TLD after **.com**.

Structure of a domain name

- **Country-code top-level domain (ccTLD)**
- A ccTLD represents a particular country or region. It consists of two letters based on the international country codes.
- To illustrate, sites from India often use the **.in** domain extension, whereas UK-based websites usually register **.uk**. Some websites from the United Kingdom use the **.co.uk** domain extension, where **.co** represents “commercial.” It’s often chosen by businesses and companies in the UK, similar to how **.com** is used in other countries.
- Here are some ccTLDs you can register:
- **.us** → United States
- **.mx** → Mexico
- **.se** → Sweden
- **.de** → Germany
- **.it** → Italy

Structure of a domain name

- **Sponsored top-level domain**
- A private entity sponsors this type of TLD to serve a specific community.
- Due to its exclusivity, there are only a handful of sponsored top-level domains available, including:
- **.gov** – sponsored by the US General Services Administration, **.gov** can only be used by US government agencies.
- **.edu** – exclusive for accredited institutions of higher education in the US.
- **.mil** – reserved for the US Department of Defense websites.
- **.travel** – restricted to businesses in the travel industry, including airlines, hotels, and restaurants.

Structure of a domain name

- **Second-level domain**
- A second-level domain (SLD) is located just before the extension. In **www.adelekeuniversity.edu.ng**, for example, “adelekeuniversity” is the SLD.
- This is the only customizable part of the domain and often the most memorable. For branding purposes, businesses should use their brand name as the SLD.
- But what if my desired domain name is already registered? In that case, you can search for the domain’s owner and offer to purchase the existing web address.

Structure of a domain name

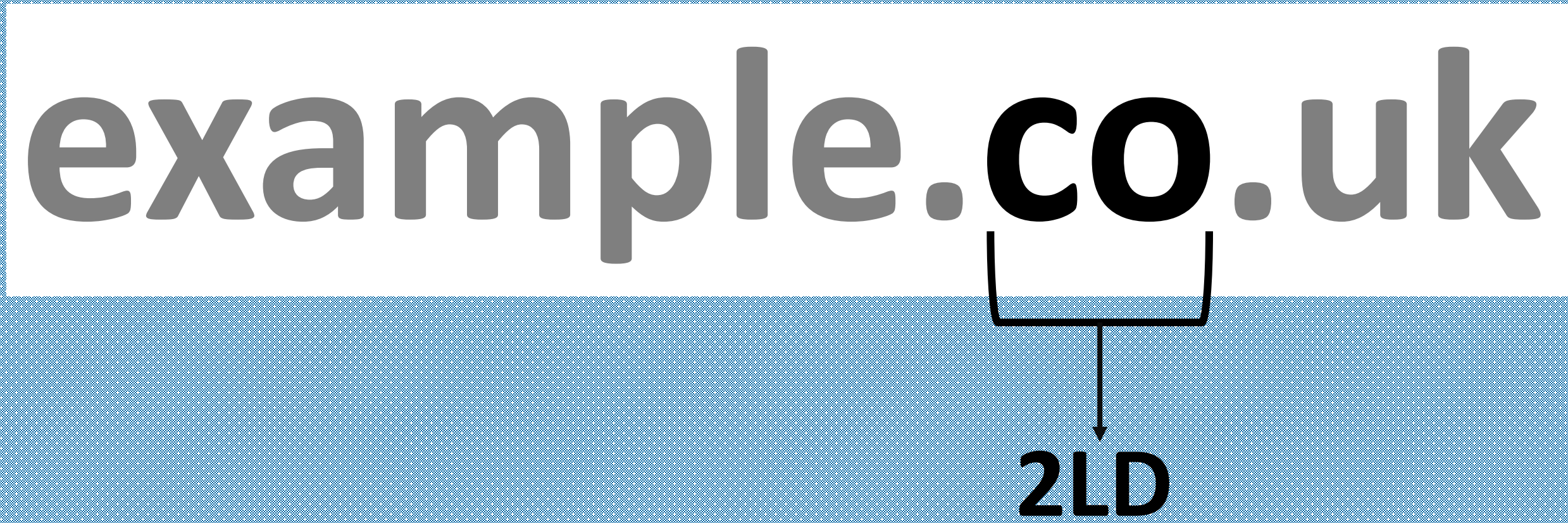
- **Third-level domain**

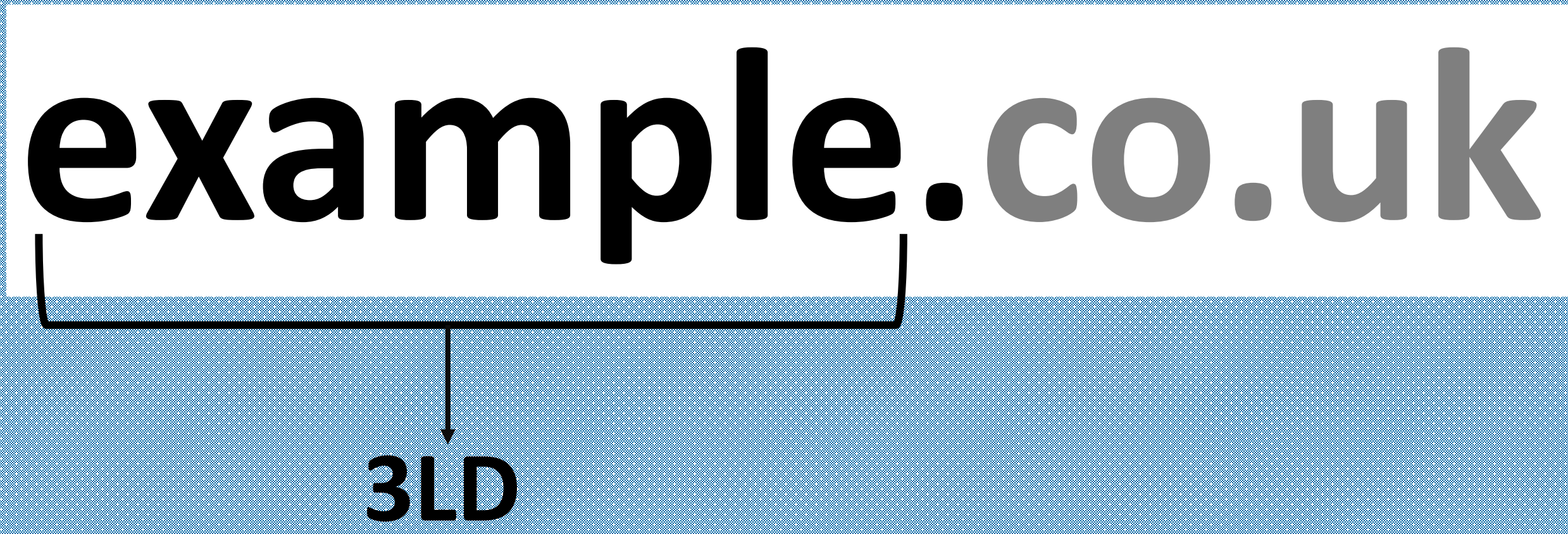
- A third-level domain, better known as a **subdomain**, refers to the part that comes before the main domain. The **www** part in most URLs is technically a subdomain, suggesting that a website is part of the World Wide Web.
- These days, **www** isn't needed anymore. Most websites work just fine without it, and a lot of them just redirect it to the main domain, like **example.com**, to keep things simple. With so many people browsing on their phones and using apps, **www** has become even less necessary.
- However, some websites also have additional subdomains besides **www** to organize content. For example, Google uses **developers.google.com** to provide information specifically for developers.

Structure of a domain name

- **Third-level domain**
- A subdomain is also useful for localization. Wikipedia has subdomains for different languages—**en.wikipedia.org** for the English version and **de.wikipedia.org** for German-speaking users, among other versions.
- Here are other common subdomain examples for a business website:
- **blog.yourwebsite.com** – for your blog section, which may cover company updates and customer stories.
- **news.yourwebsite.com** – for the company's latest news.
- **resources.yourwebsite.com** – for case studies, white papers, or eBooks.
- **support.yourwebsite.com** – for customer support, documentation, FAQs, and contact information.


example.co.uk
TLD


example.co.uk
2LD


example.co.uk
3LD

HTML

HYPERTEXT MARKUP LANGUAGE

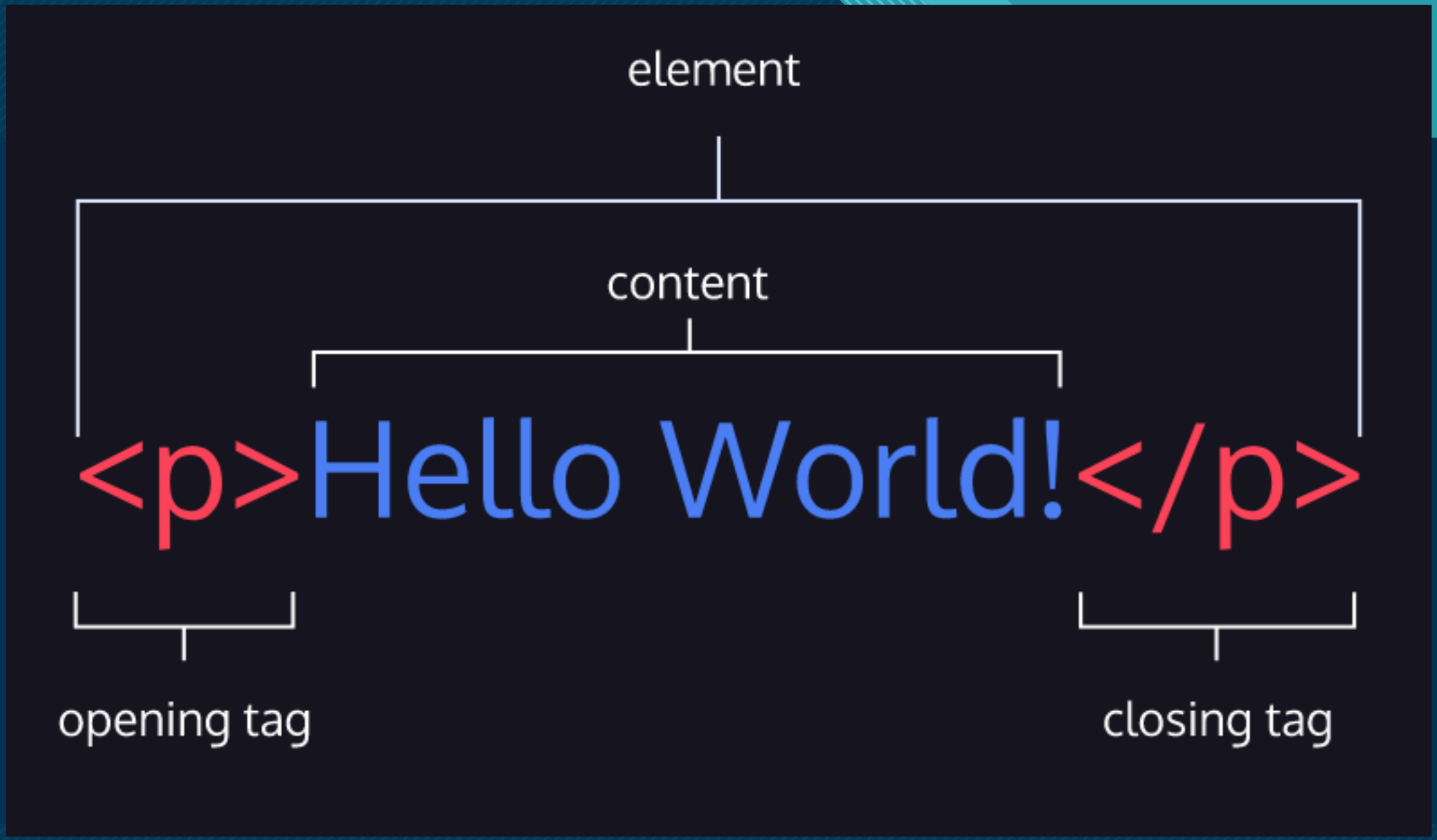
HTML

- HTML stands for Hyper Text Markup Language. It is a formatting language used to define the appearance and contents of a web page. It allows us to organize text, graphics, audio, and video on a web page.
- HTML is composed of elements. These elements structure the webpage and define its content
- The word markup refers to the symbols that are used to define structure of the text. The markup symbols tells the browser how to display the text and are often called tags.

HTML Versions

The following table shows the various versions of HTML:

Version	Year
HTML 1.0	1991
HTML 2.0	1995
HTML 3.2	1997
HTML 4.0	1999
XHTML	2000
HTML5	2012


element
content
Hello World!
opening tag
closing tag

All HTML documents must start with a document type declaration: <!DOCTYPE html>.

The HTML document itself begins with <html> and ends with </html>.

The visible part of the HTML document is between <body> and </body>.

The Body

One of the key HTML elements we use to build a webpage is the body element. Only content inside the opening and closing body tags can be displayed on the screen. Here's what opening and closing body tags look like:

<body> </body>

Once the file has a body, many different types of content – including text, images, and buttons – can be added to the body.

```
<body>
   <p>First Paragraph</p>
</body>
```

HTML Structure

HTML is organized as a collection of family tree relationships. When an element is contained inside another element, it is considered the *child* of that element. The child element is said to be *nested* inside of the *parent* element.

```
<body>
  <p>This paragraph is a child of the body</p>
</body>
```

In the example above, the <p> element is nested inside the <body> element. The <p> element is considered a child of the <body> element, and the <body> element is considered the parent. Since there can be multiple levels of nesting, this analogy can be extended to grandchildren, great-grandchildren, and beyond. The relationship between elements and their ancestor and descendent elements is known as *hierarchy*.

```
<body>
  <div>
    <h1>Sibling to p, but also grandchild of
body</h1>
    <p>Sibling to h1, but also grandchild of
body</p>
  </div>
</body>
```

In this example, the <body> element is the parent of the element. Both the <h1> and <p> elements are children of the element. Because the <h1> and <p> elements are at the same level, they are considered siblings and are both grandchildren of the <body> element.

Understanding HTML hierarchy is important because child elements can inherit behavior and styling from their parent element. You'll understand web page hierarchy when you start digging into CSS.

Headings

Headings in HTML are similar to headings in other types of media. For example, in newspapers, large headings are typically used to capture a reader's attention. Other times, headings are used to describe content, like the title of a movie or an educational article. HTML follows a similar pattern. In HTML, there are six different headings, or heading elements. Headings can be used for a variety of purposes, like titling sections, articles, or other forms of content.

The following is the list of heading elements available in HTML. They are ordered from largest to smallest in size.

1. <h1> — used for main headings. All other smaller headings are used for subheadings.
2. <h2>
3. <h3>
4. <h4>
5. <h5>
6. <h6>

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```

- The `<!DOCTYPE html>` declaration defines that this document is an HTML5 document
- The `<html>` element is the root element of an HTML page
- The `<head>` element contains meta information about the HTML page
- The `<title>` element specifies a title for the HTML page (which is shown in the browser's title bar or in the page's tab)
- The `<body>` element defines the document's body, and is a container for all the visible contents, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.
- The `<h1>` element defines a large heading
- The `<p>` element defines a paragraph

Visualization of an HTML page structure

```
<html>
  <head>
    <title>Page title</title>
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
    <p>This is another paragraph.</p>
  </body>
</html>
```

Divs

One of the most popular elements in HTML is the element. is short for “division” or a container that divides the page into sections. These sections are very useful for grouping elements in your HTML together.

```
<body>
  <div>
    <h1>Why use divs?</h1>
    <p>Great for grouping elements!</p>
  </div>
</body>
```

’s don’t inherently have a visual representation, but they are very useful when we want to apply custom styles to our HTML elements. s allow us to group HTML elements to apply the same styles for all HTML elements inside. We can also style the element as a whole.

Element	Meaning	Purpose
<b>	Bold	Highlight important information
	Strong	Similarly to bold, to highlight key text
<i>	Italic	To denote text
	Emphasised Text	Usually used as image captions
<mark>	Marked Text	Highlight the background of the text
<small>	Small Text	To shrink the text
<strike>/ <s>	Striked Out Text	To place a horizontal line across the text
<u>	Underlined Text	Used for links or text highlights
<ins>	Inserted Text	Displayed with an underline to show an inserted text
<sub>	Subscript Text	Typographical stylistic choice
<sup>	Superscript Text	Another typographical presentation style

HTML Horizontal Rules

- The <hr> tag defines a thematic break in an HTML page, and is most often displayed as a horizontal rule.
- The <hr> element is used to separate content (or define a change) in an HTML page:
- The <hr> tag is an empty tag, which means that it has no end tag.

Example

```
<h1>This is heading 1</h1>
<p>This is some text.</p>
<hr>
<h2>This is heading 2</h2>
<p>This is some other text.</p>
<hr>
```

HTML Line Breaks

- The HTML
 element defines a line break.
- Use
 if you want a line break (a new line) without starting a new paragraph:

Example

```
<p>This is<br>a paragraph<br>with line breaks.</p>
```

The HTML <pre> Element

- The HTML <pre> element defines preformatted text.
- The text inside a <pre> element is displayed in a fixed-width font (usually Courier), and it preserves both spaces and line breaks:

Example

```
<pre>
  My Bonnie lies over the ocean.

  My Bonnie lies over the sea.

  My Bonnie lies over the ocean.

  Oh, bring back my Bonnie to me.
</pre>
```

HTML Styles

- The HTML style attribute is used to add styles to an element, such as color, font, size, and more.

Example

I am Red

I am Blue

I am Big

The HTML Style Attribute

- Setting the style of an HTML element, can be done with the style attribute.

- The HTML style attribute has the following syntax:

```
<tagname style="property:value;">
```

- **The property is a CSS property. The value is a CSS value.**

Background Color

- The CSS background-color property defines the background color for an HTML element.
- Example
- Set the background color for a page to powderblue:

```
<body style="background-color:powderblue;">

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
```

Example

Set background color for two different elements:

```
<body>

<h1 style="background-color:powderblue;">This is a heading</h1>
<p style="background-color:tomato;">This is a paragraph.</p>

</body>
```

Text Color

- The CSS color property defines the text color for an HTML element:

```
<h1 style="color:blue;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>
```

Fonts

- The CSS font-family property defines the font to be used for an HTML element:

```
<h1 style="font-family:verdana;">This is a heading</h1>
<p style="font-family:courier;">This is a paragraph.</p>
```

Text Size

- The CSS font-size property defines the text size for an HTML element:

```
<h1 style="font-size:300%;">This is a heading</h1>
<p style="font-size:160%;">This is a paragraph.</p>
```

- Text Alignment
- The CSS text-align property defines the horizontal text alignment for an HTML element:

```
<h1 style="text-align:center;">Centered Heading</h1>
<p style="text-align:center;">Centered paragraph.</p>
```

HTML Quotation and Citation Elements

- We will go through the <blockquote>,<q>, <abbr>, <address>, <cite>, and <bdo> HTML elements.
- HTML <blockquote> for Quotations
- The HTML <blockquote> element defines a section that is quoted from another source.
- Browsers usually indent <blockquote> elements.

```
<p>Here is a quote from WWF's website:</p>
<blockquote cite="http://www.worldwildlife.org/who/index.html">
For 60 years, WWF has worked to help people and nature thrive. As the world's
leading conservation organization, WWF works in nearly 100 countries. At every
level, we collaborate with people around the world to develop and deliver
innovative solutions that protect communities, wildlife, and the places in which
they live.
</blockquote>
```

HTML <q> for Short Quotations

- The HTML <q> tag defines a short quotation.
- Browsers normally insert quotation marks around the quotation.

```
<p>WWF's goal is to: <q>Build a future where people live in harmony with nature.</q></p>
```

HTML <abbr> for Abbreviations

- The HTML <abbr> tag defines an abbreviation or an acronym, like "HTML", "CSS", "Mr.", "Dr.", "ASAP", "ATM".
- Marking abbreviations can give useful information to browsers, translation systems and search-engines.

- Tip: Use the global title attribute to show the description for the abbreviation/acronym when you mouse over the element.

```
<p>The <abbr title="World Health Organization">WHO</abbr> was founded in 1948.</p>
```

HTML <address> for Contact Information

- The HTML <address> tag defines the contact information for the author/owner of a document or an article.
- The contact information can be an email address, URL, physical address, phone number, social media handle, etc.
- The text in the <address> element usually renders in italic, and browsers will always add a line break before and after the <address> element.

```
<address>
Written by John Doe.<br>
Visit us at:<br>
Example.com<br>
Box 564, Disneyland<br>
USA
</address>
```

HTML <cite> for Work Title

- The HTML <cite> tag defines the title of a creative work (e.g. a book, a poem, a song, a movie, a painting, a sculpture, etc.).
- Note: A person's name is not the title of a work.
- The text in the <cite> element usually renders in italic.

```
<p><cite>The Scream</cite> by Edvard Munch. Painted in 1893.</p>
```

HTML <bdo> for Bi-Directional Override

- BDO stands for Bi-Directional Override.
- The HTML <bdo> tag is used to override the current text direction:

```
<bdo dir="rtl">This text will be written from right to left</bdo>
```

Attributes

If we want to expand an element's tag, we can do so using an attribute. Attributes are content added to the opening tag of an element and can be used in several different ways, from providing information to changing styling. Attributes are made up of the following two parts:

- The *name* of the attribute
- The *value* of the attribute

One commonly used attribute is the id. We can use the id attribute to specify different content (such as) and is really helpful when you use an element more than once. ids have several different purposes in HTML, but for now, we'll focus on how they can help us identify content on our page.

When we add an id to a , we place it in the opening tag:

```
<div id="intro">
  <h1>Introduction</h1>
</div>
```

The href Attribute

The <a> tag defines a hyperlink. The href attribute specifies the URL of the page the link goes to:

```
<a href="https://www.w3schools.com">Visit W3Schools</a>
```

The src Attribute

The <img> tag is used to embed an image in an HTML page. The src attribute specifies the path to the image to be displayed: `<img src="img_girl.jpg">`

The width and height Attributes

The <img> tag should also contain the width and height attributes, which specify the width and height of the image (in pixels):

```
<img src="img_girl.jpg" width="500" height="600">
```

The alt Attribute

The required alt attribute for the <img> tag specifies an alternate text for an image, if the image for some reason cannot be displayed. This can be due to a slow connection, or an error in the src attribute, or if the user uses a screen reader:

```
<img src="img_girl.jpg" alt="Girl with a jacket">
```

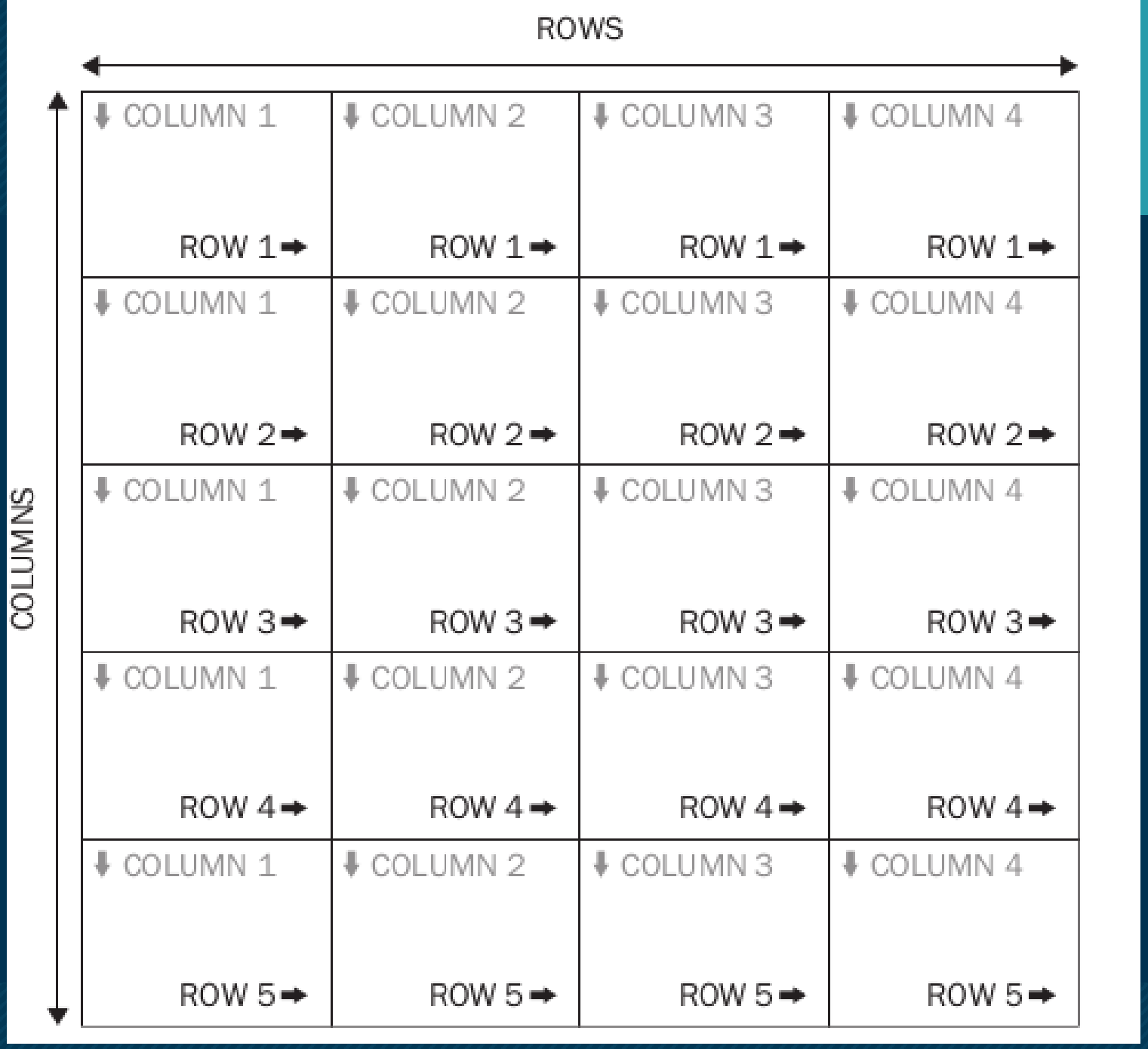
HTML Tables

ROWS

Company	Contact	Country
Alfreds Futterkiste	Maria Anders	Germany
Centro comercial Moctezuma	Francisco Chang	Mexico
Ernst Handel	Roland Mendel	Austria
Island Trading	Helen Bennett	UK
Laughing Bacchus Winecellars	Yoshi Tannamuri	Canada
Magazzini Alimentari Riuniti	Giovanni Rovelli	Italy

COLUMNS

CELLS


ROWS
COLUMNS
COLUMN 1
COLUMN 2
COLUMN 3
COLUMN 4
ROW 1
ROW 2
ROW 3
ROW 4
ROW 5

Tag	Description
<table>	Defines a table
<th>	Defines a header cell in a table
<tr>	Defines a row in a table
<td>	Defines a cell in a table
<caption>	Defines a table caption
<colgroup>	Specifies a group of one or more columns in a table for formatting
<col>	Specifies column properties for each column within a <colgroup> element
<thead>	Groups the header content in a table
<tbody>	Groups the body content in a table
<tfoot>	Groups the footer content in a table

	Outgoings($)	Receipts($)	Profit($)
Quarter 1 (Jan - Mar)	11200.00	21800.00	**10600.00**
Quarter 2 (Apr - Jun)	11700.00	22500.00	**10800.00**
Quarter 3 (Jul - Sep)	11650.00	22100.00	**10450.00**
Quarter 4 (Oct - Dec)	11850.00	22900.00	**11050.00**

The two attributes colspan and rowspan are used when you want to create fancy tables.

Colspan is short for "column span". **colspan is used in the <td> tag to specify how many columns the cell should span**:

```
<table border="1">
  <tr>
    <td colspan="3">Cell 1</td>
  </tr>
  <tr>
    <td>Cell 2</td>
    <td>Cell 3</td>
    <td>Cell 4</td>
  </tr>
</table>
```

Cell 1		
Cell 2	Cell 3	Cell 4

```
<table border="1">
  <tr>
    <td colspan="2">Cell 1</td>
    <td>Cell 2</td>
  </tr>
  <tr>
    <td>Cell 3</td>
    <td>Cell 4</td>
    <td>Cell 5</td>
  </tr>
</table>
```

Cell 1		Cell 2
Cell 3	Cell 4	Cell 5

ROWSPAN

```
<table border="1">
 <tr>
   <td rowspan="3">Cell 1</td>
   <td>Cell 2</td>
 </tr>
 <tr>
   <td>Cell 3</td>
 </tr>
 <tr>
   <td>Cell 4</td>
 </tr>
</table>
```

Cell 1	Cell 2
	Cell 3
	Cell 4

HTML FORMS

An HTML form is used to collect user input. The user input is most often sent to a server for processing

The <form> Element

The HTML **<form>** element is used to create an HTML form for user input:

The **<form>** element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc..

The <input> Element

The HTML **<input>** element is the most used form element.

An **<input>** element can be displayed in many ways, depending on the type attribute.

Type	Description
<input type="text">	Displays a single-line text input field
<input type="radio">	Displays a radio button (for selecting one of many choices)
<input type="checkbox">	Displays a checkbox (for selecting zero or more of many choices)
<input type="submit">	Displays a submit button (for submitting the form)
<input type="button">	Displays a clickable button

HTML FORM ATTRIBUTES

Attribute	Description
accept-charset	Specifies the character encodings used for form submission
action	Specifies where to send the form-data when a form is submitted
autocomplete	Specifies whether a form should have autocomplete on or off
enctype	Specifies how the form-data should be encoded when submitting it to the server (only for method="post")
method	Specifies the HTTP method to use when sending form-data
name	Specifies the name of the form
novalidate	Specifies that the form should not be validated when submitted
rel	Specifies the relationship between a linked resource and the current document
target	Specifies where to display the response that is received after submitting the form

HTML FORM ELEMENTS

<form>	Defines an HTML form for user input
<input>	Defines an input control
<textarea>	Defines a multiline input control (text area)
<label>	Defines a label for an <input> element
<fieldset>	Groups related elements in a form
<legend>	Defines a caption for a <fieldset> element
<select>	Defines a drop-down list
<optgroup>	Defines a group of related options in a drop-down list
<option>	Defines an option in a drop-down list
<button>	Defines a clickable button
<datalist>	Specifies a list of pre-defined options for input controls
<output>	Defines the result of a calculation

HTML Links

- HTML Links - Syntax
- The HTML <a> tag defines a hyperlink. It has the following syntax:

```
<a href="url">link text</a>
```

- The most important attribute of the <a> element is the href attribute, which indicates the link's destination.
- The link text is the part that will be visible to the reader.
- Clicking on the link text, will send the reader to the specified URL address.

- Example
- This example shows how to create a link to W3Schools.com:

```
<a href="https://www.w3schools.com/">Visit W3Schools.com!</a>
```

HTML Links

- By default, links will appear as follows in all browsers:
  - An unvisited link is underlined and blue
  - A visited link is underlined and purple
  - An active link is underlined and red

HTML Links - The target Attribute

By default, the linked page will be displayed in the current browser window. To change this, you must specify another target for the link. The `target` attribute specifies where to open the linked document. The `target` attribute can have one of the following values:

- `_self` - Default. Opens the document in the same window/tab as it was clicked
- `_blank` - Opens the document in a new window or tab
- `_parent` - Opens the document in the parent frame
- `_top` - Opens the document in the full body of the window

```
<a href="https://www.w3schools.com/" target="_blank">Visit W3Schools!</a>
```

Absolute URLs vs. Relative URLs

- The previous example is an absolute URL (a full web address) in the href attribute.
- A local link (a link to a page within the same website) is specified with a relative URL (without the "https://www" part):

```
<h2>Absolute URLs</h2>
<p><a href="https://www.w3.org/">W3C</a></p>
<p><a href="https://www.google.com/">Google</a></p>

<h2>Relative URLs</h2>
<p><a href="html_images.asp">HTML Images</a></p>
<p><a href="/css/default.asp">CSS Tutorial</a></p>
```

HTML Links - Use an Image as a Link

- To use an image as a link, just put the <img> tag inside the <a> tag:

```
<a href="default.asp">
<img src="smiley.gif" alt="HTML tutorial" style="width:42px;height:42px;">
</a>
```

Link to an Email Address

- Use mailto: inside the href attribute to create a link that opens the user's email program (to let them send a new email):

```
<a href="mailto:someone@example.com">Send email</a>
```

Button as a Link

To use an HTML button as a link, you have to add some JavaScript code.

JavaScript allows you to specify what happens at certain events, such as a click of a button:

```
<button onclick="document.location='default.asp'">HTML Tutorial</button>
```

Link Titles

The title attribute specifies extra information about an element. The information is most often shown as a tooltip text when the mouse moves over the element.

```
<a href="https://www.w3schools.com/html/" title="Go to W3Schools HTML
section">Visit our HTML Tutorial</a>
```

HTML Images Syntax

- The HTML <img> tag is used to embed an image in a web page.

- Images are not technically inserted into a web page; images are linked to web pages. The <img> tag creates a holding space for the referenced image.
- The <img> tag is empty, it contains attributes only, and does not have a closing tag.
- The <img> tag has two required attributes:
  - src - Specifies the path to the image
  - alt - Specifies an alternate text for the image

Syntax

```
<img src="url" alt="alternatetext">
```

The src Attribute

- The required src attribute specifies the path (URL) to the image.

- Note: When a web page loads, it is the browser, at that moment, that gets the image from a web server and inserts it into the page. Therefore, make sure that the image actually stays in the same spot in relation to the web page, otherwise your visitors will get a broken link icon. The broken link icon and the alt text are shown if the browser cannot find the image.

```
<img src="img_chania.jpg" alt="Flowers in Chania">
```

The alt Attribute

- The required alt attribute provides an alternate text for an image, if the user for some reason cannot view it (because of slow connection, an error in the src attribute, or if the user uses a screen reader).

- The value of the alt attribute should describe the image:

```
<img src="img_chania.jpg" alt="Flowers in Chania">
```

- If a browser cannot find an image, it will display the value of the alt attribute

Image Size - Width and Height

- You can use the style attribute to specify the width and height of an image.

```
<img src="img_girl.jpg" alt="Girl in a jacket" style="width:500px;height:600px;">
```

- Alternatively, you can use the width and height attributes:

```
<img src="img_girl.jpg" alt="Girl in a jacket" width="500" height="600">
```

- **NOTE: The width and height attributes always define the width and height of the image in pixels.**

Common Image Formats

Abbreviation	File Format	File Extension
APNG	Animated Portable Network Graphics	.apng
GIF	Graphics Interchange Format	.gif
ICO	Microsoft Icon	.ico, .cur
JPEG	Joint Photographic Expert Group image	.jpg, .jpeg, .jfif, .pjpeg, .pjp
PNG	Portable Network Graphics	.png
SVG	Scalable Vector Graphics	.svg

HTML Background Images

- Background Image on an HTML element
- To add a background image on an HTML element, use the HTML style attribute and the CSS background-image property:

```
<p style="background-image: url('img_girl.jpg');">
```

- You can also specify the background image in the <style> element, in the <head> section:

```
<style>
p {
  background-image: url('img_girl.jpg');
}
</style>
```

Background Image on a Page

- If you want the entire page to have a background image, you must specify the background image on the <body> element:

```
<style>
body {
  background-image: url('img_girl.jpg');
}
</style>
```

Favicon

A favicon is a small image displayed next to the page title in the browser tab.



- To add a favicon to your website, either save your favicon image to the root directory of your webserver, or create a folder in the root directory called images, and save your favicon image in this folder. A common name for a favicon image is "favicon.ico".
- Next, add a <link> element to your "index.html" file, after the <title> element, like this:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page Title</title>
  <link rel="icon" type="image/x-icon" href="/images/favicon.ico">
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

ID vs CLASS

ID’s are unique

The id attribute specifies a unique id for an HTML element. The value of the id attribute must be unique within the HTML document.

The id attribute is used to point to a specific style declaration in a style sheet. It is also used by JavaScript to access and manipulate the element with the specific id.

The syntax for id is: write a hash character (#), followed by an id name. Then, define the CSS properties within curly braces {}.

```
<!DOCTYPE html>
<html>
<head>
<style>
#myHeader {
  background-color: lightblue;
  color: black;
  padding: 40px;
  text-align: center;
}
</style>
</head>
<body>

<h1 id="myHeader">My Header</h1>

</body>
</html>
```

CLASS

- The HTML class attribute is used to specify a class for an HTML element.
- Multiple HTML elements can share the same class.
- The class attribute is often used to point to a class name in a style sheet. It can also be used by a JavaScript to access and manipulate elements with the specific class name.

```
<!DOCTYPE html>
<html>
<head>
    <style>
    .city {background-color: tomato;
      color: white;
      border: 2px solid black;
      margin: 20px;
      padding: 20px;}
    </style>
</head>
<body>
    <div class="city">
      <h2>London</h2>
      <p>London is the capital of England.</p>
    </div>
<div class="city">
  <h2>Paris</h2>
  <p>Paris is the capital of France.</p>
</div>
</body>
</html>
```

BLOCK AND INLINE ELEMENTS

- A block-level element always starts on a new line, and the browsers automatically add some space (a margin) before and after the element.
- A block-level element always takes up the full width available (stretches out to the left and right as far as it can).
- Here are the block-level elements in HTML:

```
<address> <article> <aside><blockquote> <canvas> <dd> <div>

<dl> <dt> <fieldset> <figcaption> <figure> <footer> <form>

<h1>-<h6> <header> <hr> <li> <main> <nav> <noscript> <ol>

<p> <pre> <section> <table> <tfoot> <ul> <video>
```

- An inline element does not start on a new line.
- An inline element only takes up as much width as necessary.
- Here are the inline elements in HTML:

```
<a> <abbr> <acronym> <b> <bdo> <big> <br> <button> <cite>
<code> <dfn> <em> <i> <img> <input> <kbd> <label> <map>
<object> <output> <q> <samp> <script> <select> <small> <span>
<strong> <sub> <sup> <textarea> <time> <tt> <var>
```

DIV & SPAN ELEMENT

The and elements allow you to group together several elements to create sections or subsections of a page.

This is the very important block level tag which plays a big role in grouping various other HTML tags and applying CSS on group of elements. Even now tag can be used to create a web page layout where we define different parts (Left, Right, Top etc.) of the page using tag. This tag does not provide any visual change on the block but this has more meaning when it is used with CSS.

- The HTML is an inline element and it can be used to group inline-elements in an HTML document. This tag also does not provide any visual change on the block but has more meaning when it is used with CSS.
- The difference between the tag and the tag is that the tag is used with inline elements whereas the tag is used with block-level elements.

HTML LAYOUT ELEMENTS

The following tags have been introduced for better structure −

section − This tag represents a generic document or application section. It can be used together with h1-h6 to indicate the document structure.

article − This tag represents an independent piece of content of a document, such as a blog entry or newspaper article.

aside − This tag represents a piece of content that is only slightly related to the rest of the page.

header − This tag represents the header of a section.
footer − This tag represents a footer for a section and can contain information about the author, copyright information, et cetera.
nav − This tag represents a section of the document intended for navigation.
dialog − This tag can be used to mark up a conversation.
figure − This tag can be used to associate a caption together with some embedded content, such as a graphic or video.

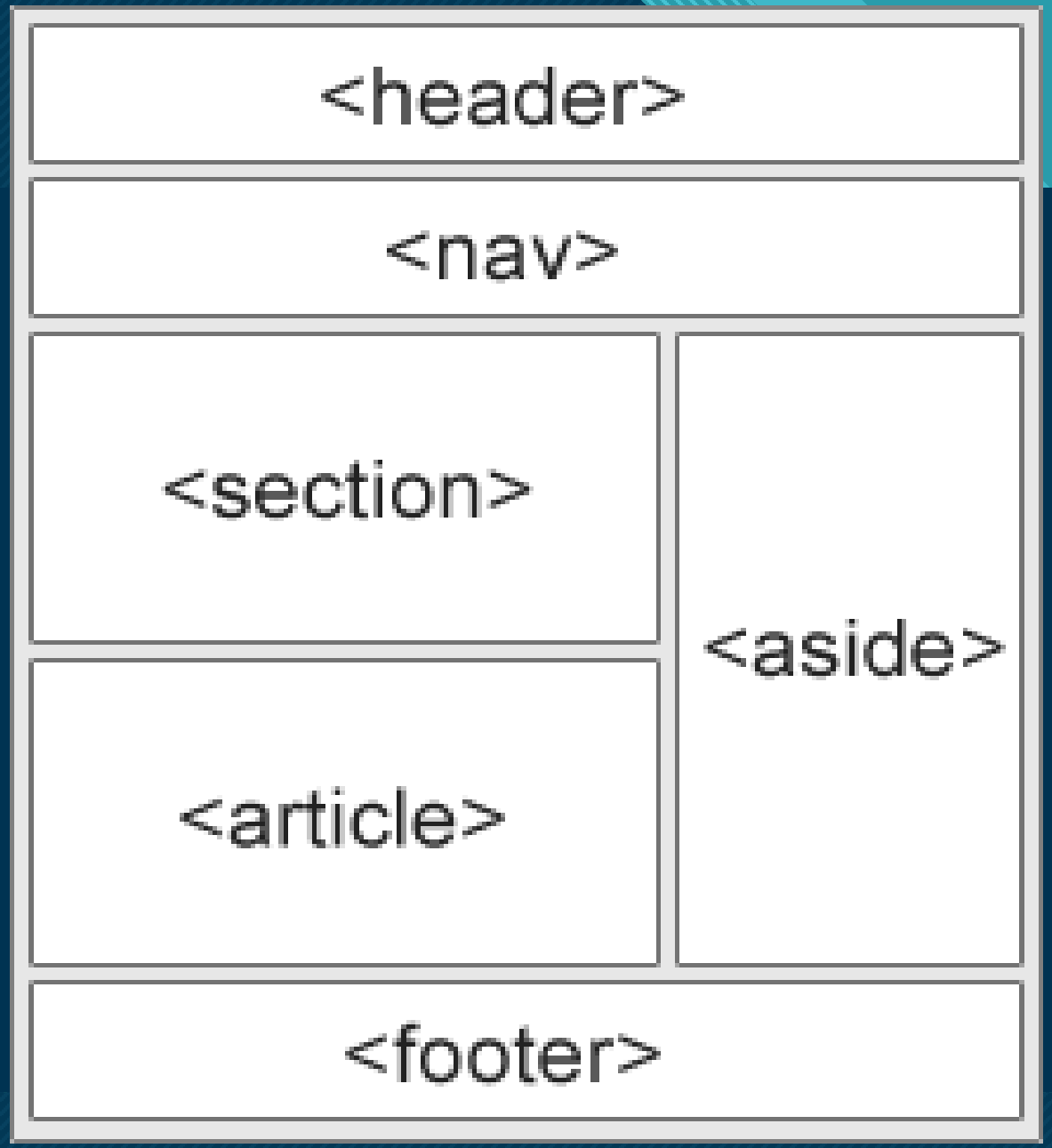
header − This tag represents the header of a section.

footer − This tag represents a footer for a section and can contain information about the author, copyright information, et cetera.

nav − This tag represents a section of the document intended for navigation.

dialog − This tag can be used to mark up a conversation.

figure − This tag can be used to associate a caption together with some embedded content, such as a graphic or video.


<header>
<nav>
<section>
<aside>
<article>
<footer>

WHAT IS CSS?

What is CSS?

CSS (Cascading Style Sheets) is a language used to create documents on the web. CSS is meant for the ***presentation*** of a web page's content, or rather, how a web page is ***meant to look.***

We can use CSS to modify any HTML tag, so as to deliver a branded experience in graphical browsers. Users accessing the pages with more primitive devices will still be able to understand and use the site. This is called ***graceful degradation.***

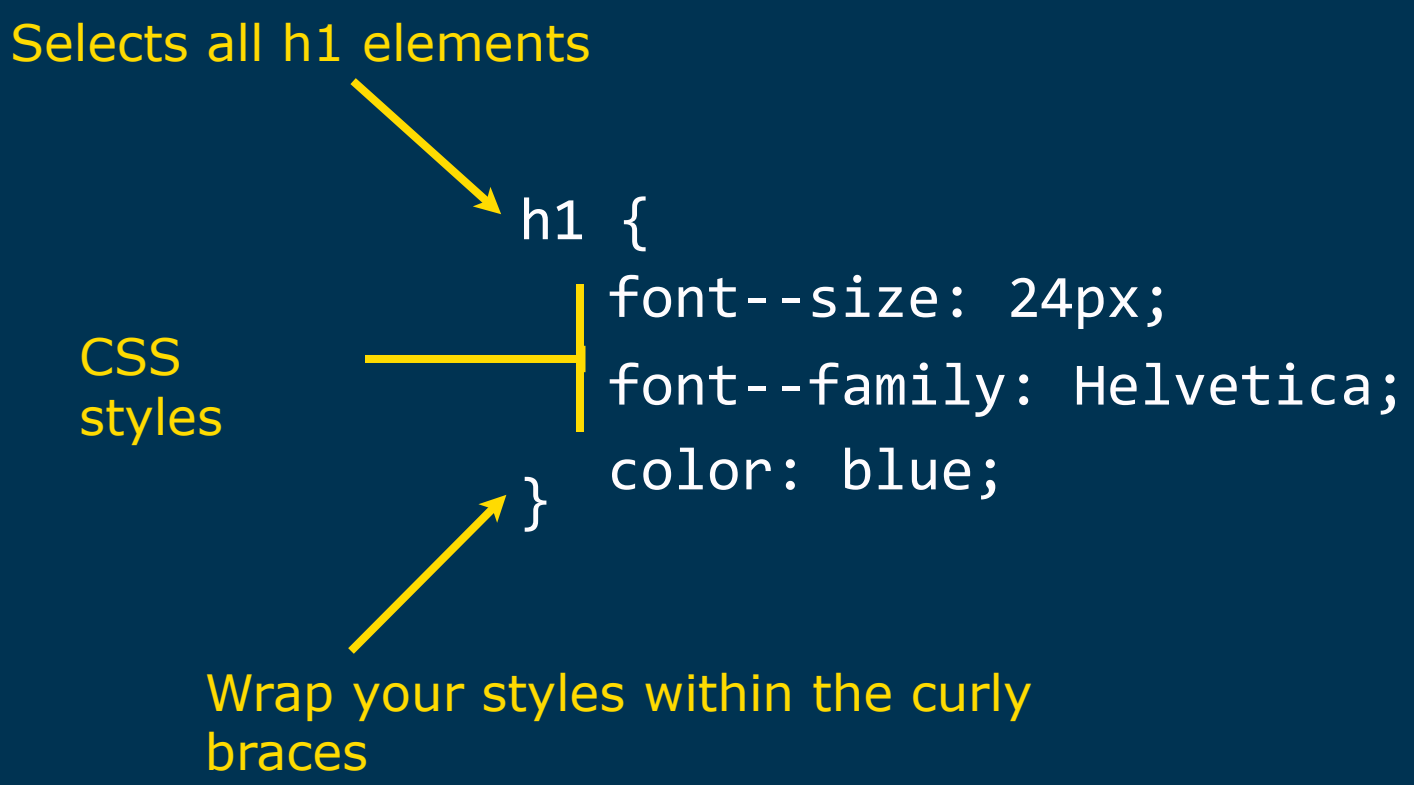
The Style Rule

CSS allows the author to specify formatting & presentation of HTML elements using CSS Style Rules.

A CSS Style Rule looks like this:

```
h1 {
   font--size: 24px;
   font--family: Helvetica;
   color: blue;
}
```

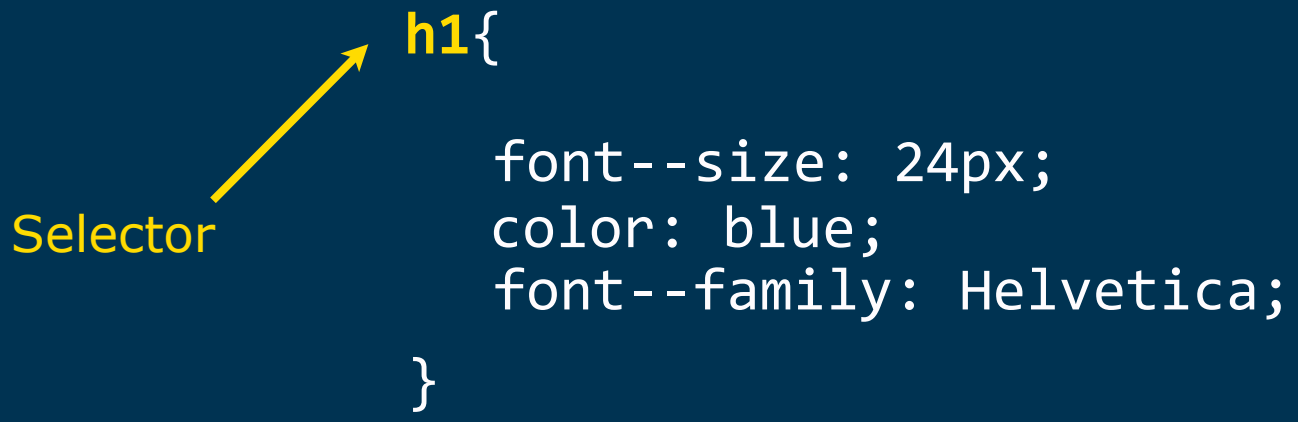
The Syntax


Selects all h1 elements
h1 {
font--size: 24px;
font--family: Helvetica;
color: blue;
}
CSS styles
Wrap your styles within the curly braces

Selectors, Properties & Values

Selectors

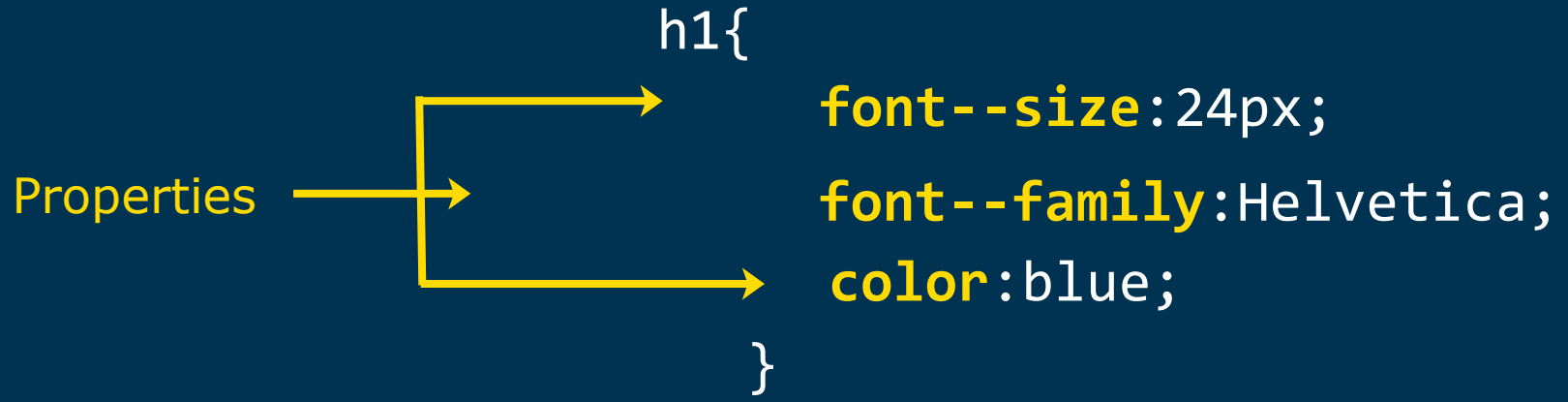
Selectors are used in CSS to style a specific HTML element, or multiple HTML elements.



Selectors, Properties & Values

Values

A value is assigned to a property.



FONT

font

font-family

font-size

font-style

font-variant

font-weight

TEXT

color

direction

letter-spacing

text-align

text-decoration

text-indent

text-shadow

text-transform

unicode-bidi

white-space

word-spacing

BACKGROUND

background

background-attachment

background-color

background-image

background-position

background-repeat

BORDER

border

border-color

border-style

border-width

border-bottom

border-bottom-color

border-bottom-style

border-bottom-width

border-left

border-left-color

border-left-style

border-left-width

border-right

border-right-color

border-right-style

border-right-width

border-top

border-top-color

border-top-style

border-top-width

MARGIN

margin

margin-bottom

margin-left

margin-right

margin-top

PADDING

padding

padding-bottom

padding-left

padding-right

padding-top

DIMENSIONS

height

line-height

max-height

max-width

min-height

min-width

width

POSITIONING

bottom

clip

left

overflow

right

top

vertical-align

z-index

OUTLINES

outline

outline-color

outline-style

outline-width

TABLE

border-collapse

border-spacing

caption-side

empty-cells

table-layout

LIST and MARKER

list-style

list-style-image

list-style-position

list-style-type

marker-offset

GENERATED CONTENT

content

counter-increment

counter-reset

quotes

CLASSIFICATION

clear

cursor

display

float

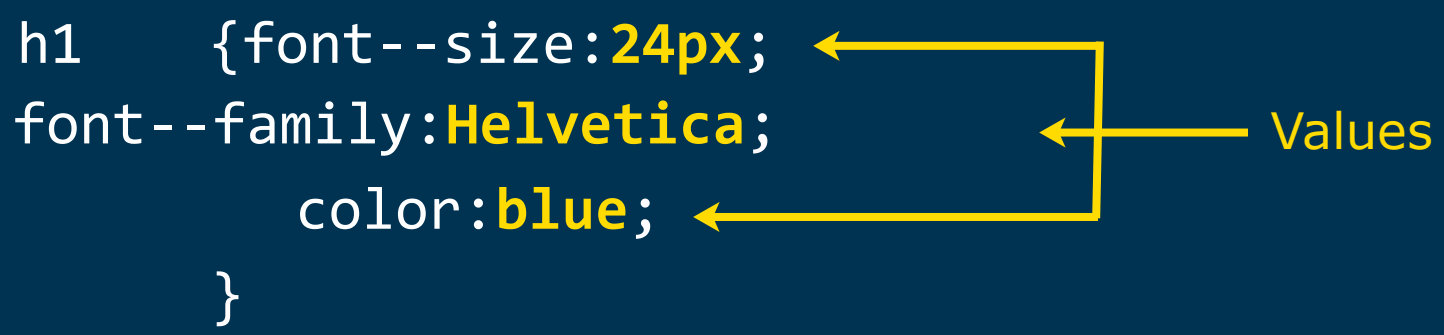
position

visibility

Selectors, Properties & Values

Values

A value is assigned to a property.



CSS selectors are used to "find" (or select) the HTML elements you want to style.

We can divide CSS selectors into five categories:

- Simple selectors (select elements based on name, id, class)
- Combinator selectors (select elements based on a specific relationship between them)
- Pseudo-class selectors (select elements based on a certain state)
- Pseudo-elements selectors (select and style a part of an element)
- Attribute selectors (select elements based on an attribute or attribute value)

The CSS element Selector

- The element selector selects HTML elements based on the element name.

Example

Here, all <p> elements on the page will be center-aligned, with a red text color:

```
p {
  text-align: center;
  color: red;
}
```

The CSS id Selector

- The id selector uses the id attribute of an HTML element to select a specific element.
- The id of an element is unique within a page, so the id selector is used to select one unique element!
- To select an element with a specific id, write a hash (#) character, followed by the id of the element.

Example

The CSS rule below will be applied to the HTML element with id="para1":

```
#para1 {
  text-align: center;
  color: red;
}
```

The CSS class Selector

- The class selector selects HTML elements with a specific class attribute.
- To select elements with a specific class, write a period (.) character, followed by the class name.

Example

In this example all HTML elements with class="center" will be red and center-aligned:

```
.center {
  text-align: center;
  color: red;
}
```

The CSS class Selector

- You can also specify that only specific HTML elements should be affected by a class.

Example

In this example only <p> elements with class="center" will be red and center-aligned:

```
p.center {
  text-align: center;
  color: red;
}
```

The CSS Universal Selector

The universal selector (*) selects all HTML elements on the page.

Example

The CSS rule below will affect every HTML element on the page:

```
* {
  text-align: center;
  color: blue;
}
```

The CSS Grouping Selector

- The grouping selector selects all the HTML elements with the same style definitions.
- Look at the following CSS code (the h1, h2, and p elements have the same style definitions):

```
h1 {
  text-align: center;
  color: red;
}

h2 {
  text-align: center;
  color: red;
}

p {
  text-align: center;
  color: red;
}
```

The CSS Grouping Selector

- It will be better to group the selectors, to minimize the code.
- To group selectors, separate each selector with a comma.

Example

In this example we have grouped the selectors from the code above:

```
h1, h2, p {
  text-align: center;
  color: red;
}
```

How To Add CSS

When a browser reads a style sheet, it will format the HTML document according to the information in the style sheet.

There are three ways of inserting a style sheet:

- External CSS
- Internal CSS
- Inline CSS

External CSS

- With an external style sheet, you can change the look of an entire website by changing just one file!
- Each HTML page must include a reference to the external style sheet file inside the <link> element, inside the head section.
- External styles are defined within the <link> element, inside the <head> section of an HTML page:

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

External CSS

- An external style sheet can be written in any text editor, and must be saved with a .css extension.
- The external .css file should not contain any HTML tags.
- Here is how the "mystyle.css" file looks:

"mystyle.css"

```css
body {
  background-color: lightblue;
}

h1 {
  color: navy;
  margin-left: 20px;
}
```

Internal CSS

- An internal style sheet may be used if one single HTML page has a unique style.
- The internal style is defined inside the <style> element, inside the head section.

Example

Internal styles are defined within the <style> element, inside the <head> section of an HTML page:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: linen;
}

h1 {
  color: maroon;
  margin-left: 40px;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

Inline CSS

- An inline style may be used to apply a unique style for a single element.
- To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.

Example

Inline styles are defined within the "style" attribute of the relevant element:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>

</body>
</html>
```

Multiple Style Sheets

- If some properties have been defined for the same selector (element) in different style sheets, the value from the last read style sheet will be used.

Assume that an **external style sheet** has the following style for the <h1> element:

```
h1 {
  color: navy;
}
```

Then, assume that an **internal style sheet** also has the following style for the <h1> element:

```
h1 {
  color: orange;
}
```

Cascading Order

What style will be used when there is more than one style specified for an HTML element?

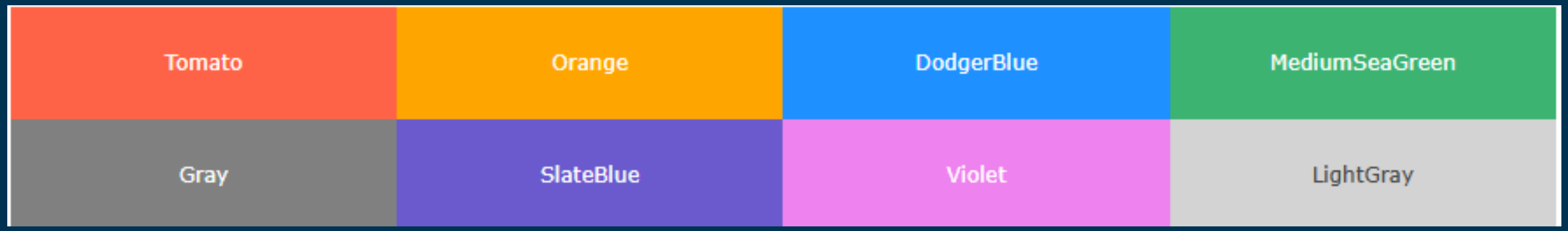
All the styles in a page will "cascade" into a new "virtual" style sheet by the following rules, where number one has the highest priority:

1. Inline style (inside an HTML element)
2. External and internal style sheets (in the head section)
3. Browser default

So, an inline style has the highest priority, and will override external and internal styles and browser defaults.

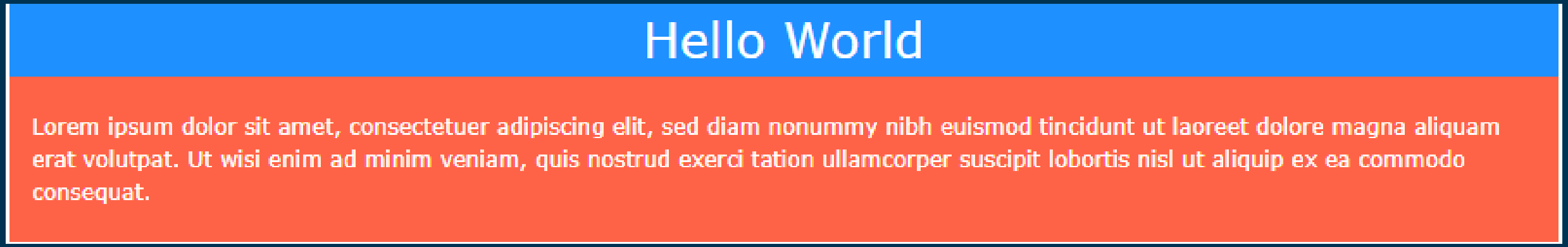
CSS Colors

In CSS, a color can be specified by using a predefined color name:



CSS Background Color

You can set the background color for HTML elements:




```
<h1 style="background-color:DodgerBlue;">Hello World</h1>
<p style="background-color:Tomato;">Lorem ipsum...</p>
```

CSS Text Color

You can set the color of text:

Hello World

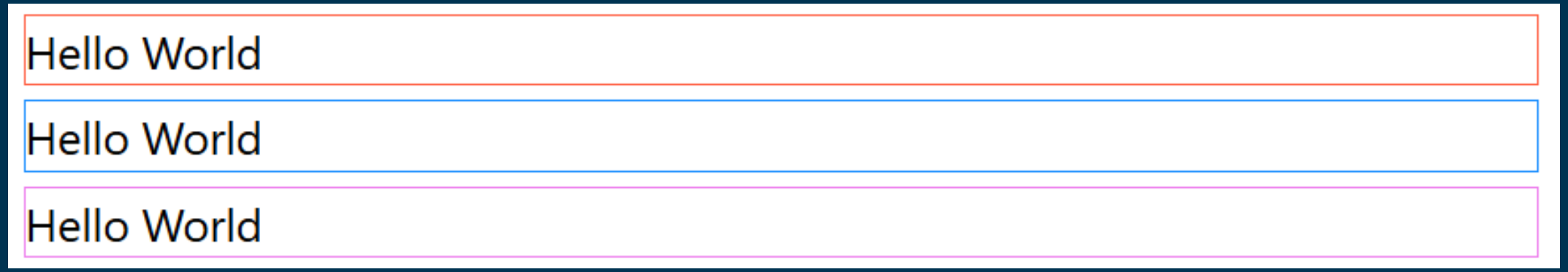
Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.

Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.

```
<h1 style="color:Tomato;">Hello World</h1>
<p style="color:DodgerBlue;">Lorem ipsum...</p>
<p style="color:MediumSeaGreen;">Ut wisi enim...</p>
```

CSS Border Color

You can set the color of borders:




```
<h1 style="border:2px solid Tomato;">Hello World</h1>
<h1 style="border:2px solid DodgerBlue;">Hello World</h1>
<h1 style="border:2px solid Violet;">Hello World</h1>
```

CSS Color Values

In CSS, colors can also be specified using RGB values, HEX values, HSL values, RGBA values, and HSLA values:

Same as color name "Tomato":

rgb(255, 99, 71)

#ff6347

hsl(9, 100%, 64%)

```
<h1 style="background-color:rgb(255, 99, 71);">...</h1>
<h1 style="background-color:#ff6347;">...</h1>
<h1 style="background-color:hsl(9, 100%, 64%);">...</h1>
```

CSS Color Values

Same as color name "Tomato", but 50% transparent:

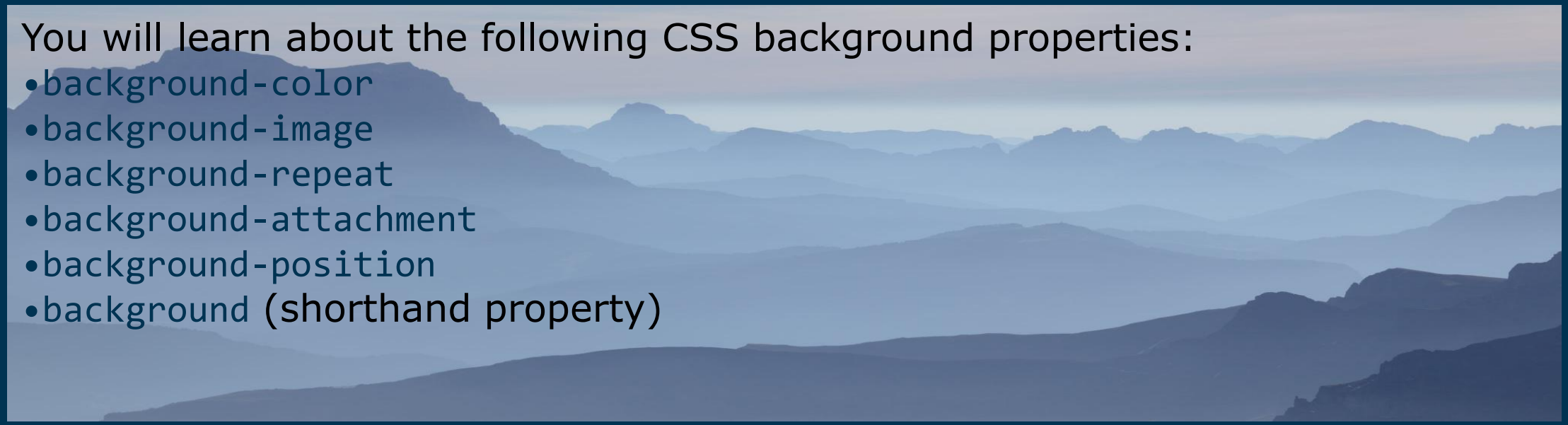
rgba(255, 99, 71, 0.5)

hsla(9, 100%, 64%, 0.5)

```
<h1 style="background-color:rgba(255, 99, 71, 0.5);">...</h1>
<h1 style="background-color:hsla(9, 100%, 64%, 0.5);">...</h1>
```

CSS Backgrounds

The CSS background properties are used to add background effects for elements.



You will learn about the following CSS background properties:

- `background-color`
- `background-image`
- `background-repeat`
- `background-attachment`
- `background-position`
- `background` (shorthand property)

CSS background-color

The `background-color` property specifies the background color of an element.

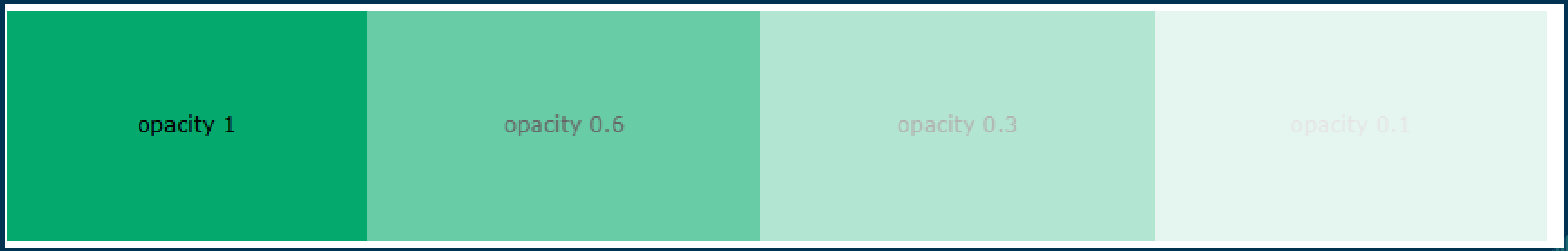
The background color of a page is set like this:

```
body {
  background-color: lightblue;
}
```

CSS background-color

Opacity / Transparency

The opacity property specifies the opacity/transparency of an element. It can take a value from 0.0 - 1.0. The lower value, the more transparent:



CSS background-color

Transparency using RGBA

If you do not want to apply opacity to child elements, like in our example above, use RGBA color values. The following example sets the opacity for the background color and not the text:




```
div {
  background: rgba(0, 128, 0, 0.3) /* Green background with 30% opacity */
}
```

CSS Background Image

The background-image property specifies an image to use as the background of an element.

By default, the image is repeated so it covers the entire element.

Set the background image for a page:

```
body {
  background-image: url("paper.gif");
}
```

CSS background-repeat

By default, the background-image property repeats an image both horizontally and vertically.

Some images should be repeated only horizontally or vertically, or they will look strange, like this:

Inheritance of Styles

Let's refer back to the parent/child relationship we learned about earlier.

```
<h1>Hey, <em>look</em> at that monkey</h1>
```

The h1 is the parent element of the em. Therefore, if we add the style color: blue; to the h1, then the em will inherit that style and also be color: blue;

Inheritance of Styles

However, if we define a separate colour for the em, say color: orange; then the em will display in that colour instead, overriding the inherited style of it's parent.

Inheritance of Styles

That’s pretty much how the inheritance of styles works. Assign a style to an element, it’s child elements will inherit the styles assigned to it’s parent, or ancestor.

However, there are exceptions. If you gave the `h1` a border, then it’s child elements would not inherit the border, because that would look awful if every child element had a border just because you wanted it’s parent to have one.

Measurements in CSS
Pixels, Percentages, Ems & Rems!

In CSS, you use measurements a lot. Margin, padding, font size, width, height, etc. all use measurements to determine the size of a specific value. For example:

```
h1 {
   font--size:2.4rem;
   width: 50%;
   padding: 2em;
   margin: 10px;
}
```

Measurements in CSS
Pixels, Percentages, Ems & Rems!

Here are the most common measurements used in CSS:

Unit	Description
`Px`	pixels
`%`	percent
`em`	Relative to the font size of the parent element. If the parent element has a font size of 14px, then 1em = 14px
`rem`	Relative to the font size of the root element (html element). If the html element is 18px, then 1rem = 18px

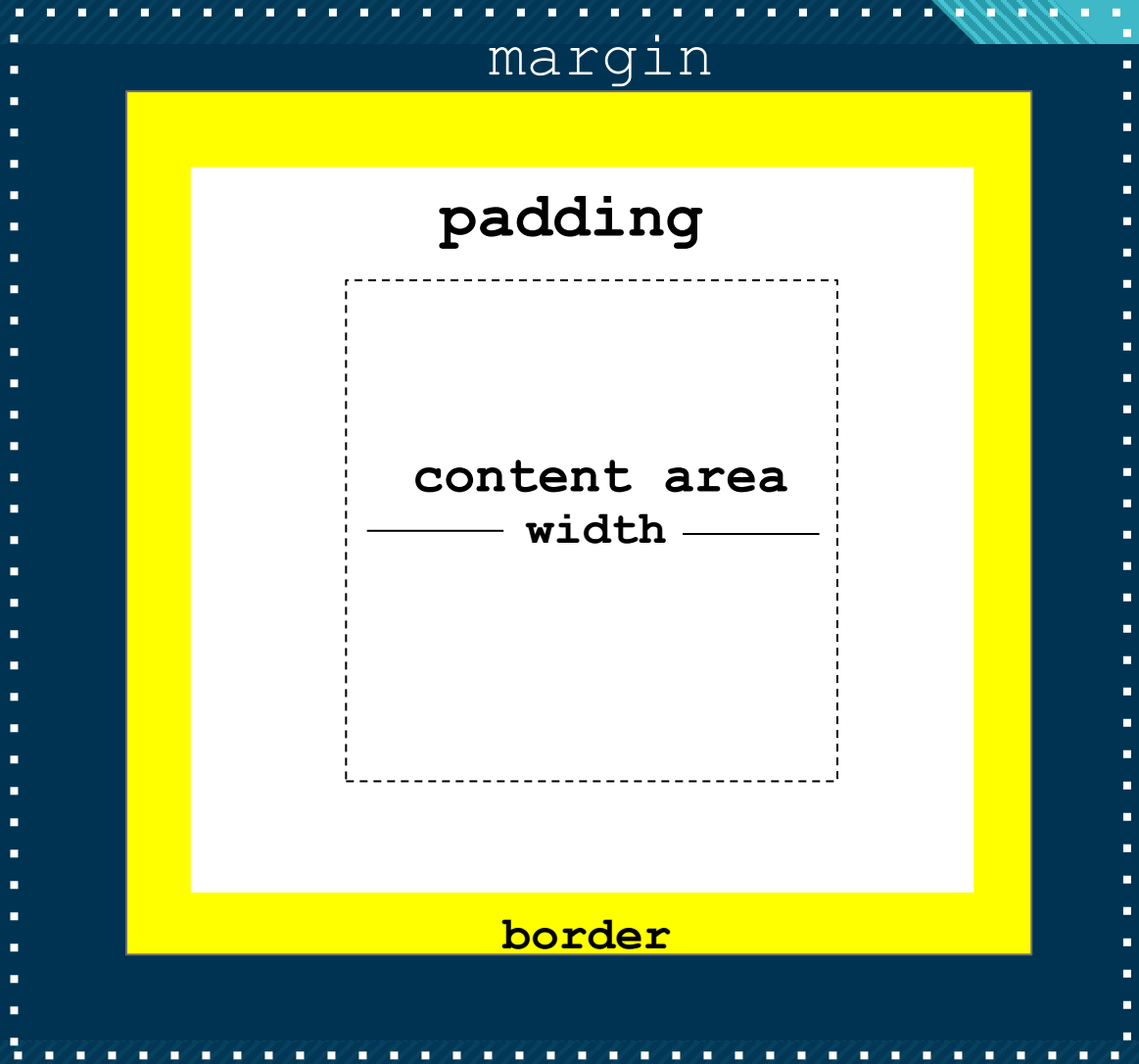
The CSS Box Model

Think of all elements in HTML as appearing within a box in the layout of a page. CSS treats all elements this way.

Both block & inline-level elements occupy a box.

All boxes have margin, padding, and borders. Block-level elements have a width property, whereas inline-level elements do not.

The CSS Box Model



Margin

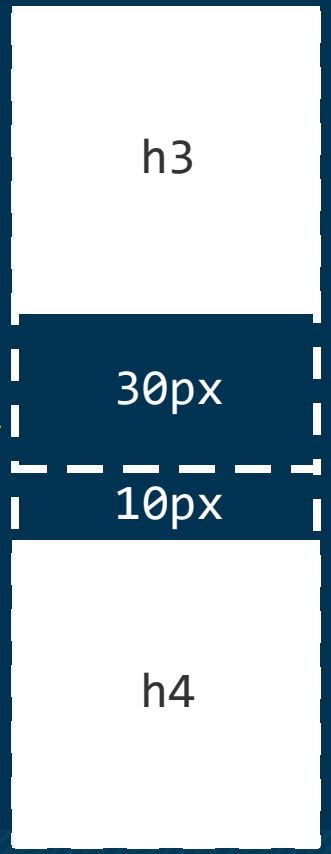
Margins live outside the box. In the example, the margin has a dashed line around it just to show the area it resides in. In the browser, margins are invisible.

Top & bottom margins overlap on adjacent elements. If you have an h3 followed by an h4, and you've defined the h3 to have margin-bottom: 30px and the h4 to have margin-top: 10px you might think this adds up to 40px but it doesn't. The two margins overlap, and the larger of the two margins are used, therefore the margin between these elements would be 30px.

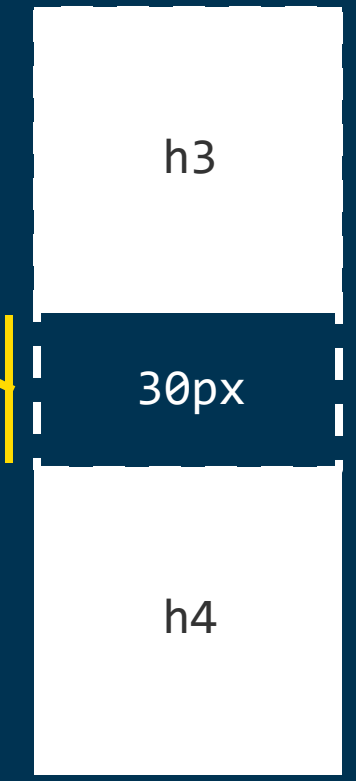
Margin

For example:

You think this would happen. But it doesn't.



The smallest margin is absorbed into the larger margin



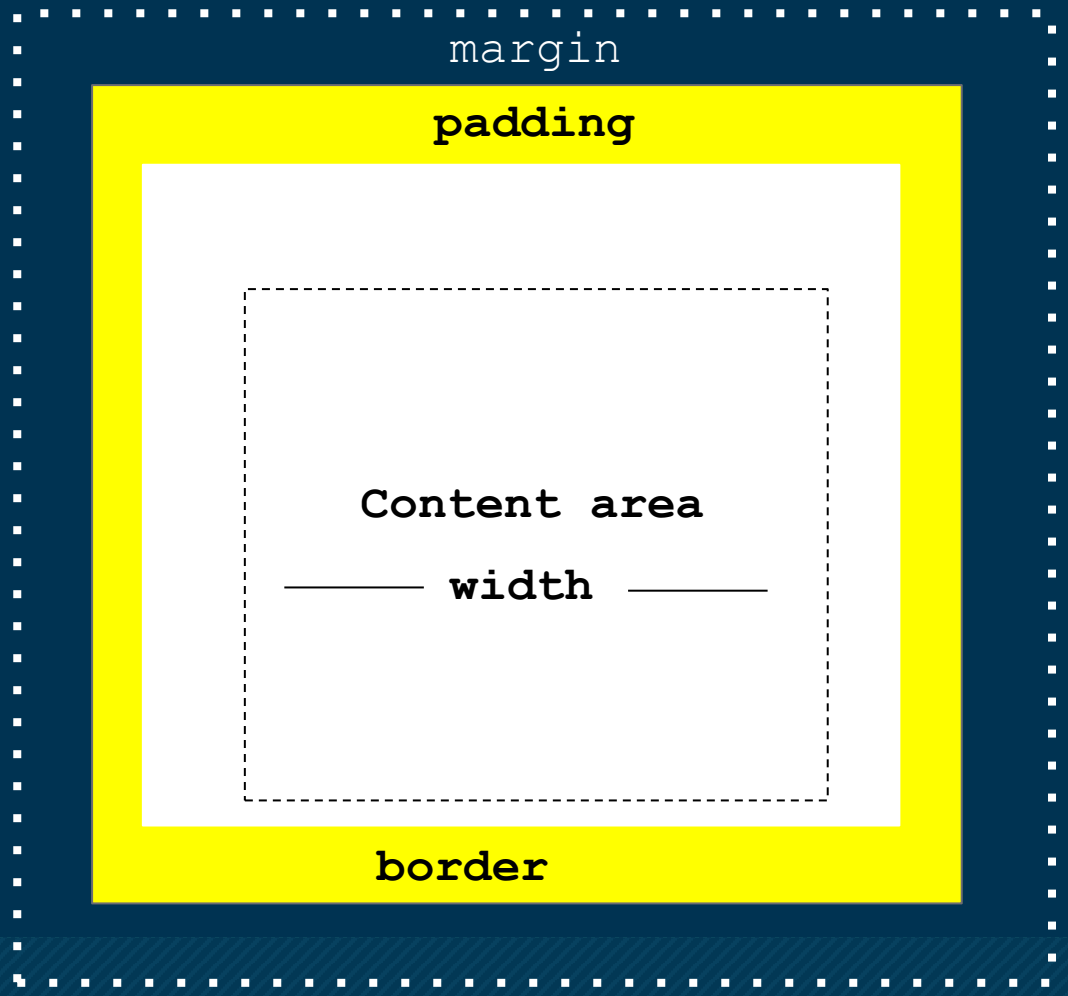
Note: This only happens on top & bottom margins, not left and right margins!

Inline Elements & Margin

Inline elements do not display vertical margins. However, they do display left and right margins.

Padding

Padding resides within the box. So, you can see that the background colour in the example is the same in the content area and the padding area.



Using Margin & Padding

To add margin & padding in CSS, all we have to do is define the size of the margin on all four sides of the box. Here is an example:

```
margin--top: 10px;
margin--right: 10px;
margin--bottom: 10px;
margin--left: 10px;
```

Using Margin & Padding

Note that all four values consist of a measurement (10) and a unit (px) with no spaces between them. In the previous example, we declared all sides of the box to have the same values. We can write the previous example in a much simpler way:

```
margin: 10px;
```

Using Margin & Padding

If any of the values are equal to `0px`, we can skip writing the unit, like so:

```
margin: 0;
```

Using Margin & Padding

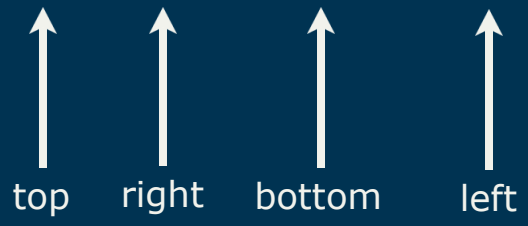
Another way of writing our margins & padding more efficiently is like so:

```
margin: 0 10px 20px 30px;
```

Using Margin & Padding

Remember the order of the values:

```
margin: 0 10px 20px 30px;
```



Using Margin & Padding

In some cases, you can use two values:

```
margin: 0 10px;
```



Using Margin & Padding

And three values:

```
margin: 0 10px 20px;
```



Using Margin & Padding

All of the previous techniques apply to defining padding and borders as well.

Borders

These guys are a little different, though. Borders don't only have a border-width, they also have a border-style property, and a border-color property. Only the border-style property is required. Here's how you declare them:

```
border--style:solid;

border--width: 4px;
border--color:#333;
```

Borders

What if all sides have the same style, width and colour? There's a shorthand for that!

```
border: solid 4px #333;
```

Borders

You can also create variations in the border properties by overriding some properties with more specific declarations:

```
border: solid 4px #333;
border--bottom--color:#fc3;
```

Since the second border declaration comes later in the stylesheet than the first declaration, the CSS overrides the bottom border colour of #333 with #fc3 instead.

PROPERTY	PURPOSE
`font`	Enables you to combine several of the following properties into one.
`font-family`	Specifies the typeface or family of the font that should be used.
`font-size`	Specifies the size of the font.
`font-weight`	Specifies whether the font should be normal or bold.
`font-style`	Specifies whether the font should be normal, italic, or oblique.
`font-variant`	Specifies whether the font should be normal or small caps.

CSS Specificity

As you get deeper into coding a website, your CSS files become large. *Sometimes very large!* The larger they become, the higher the chance you have of running into problems.

The Specificity Rule

More Specific = Higher Priority

For Example

Because CSS means, Cascading Style Sheets, the styles "cascade" down the document. Check this out:

```
h1 { background: black; }
h1 { background: white; }
```

In this example, the latter style is applied to the `h1` because it comes later in the document.

Another Example

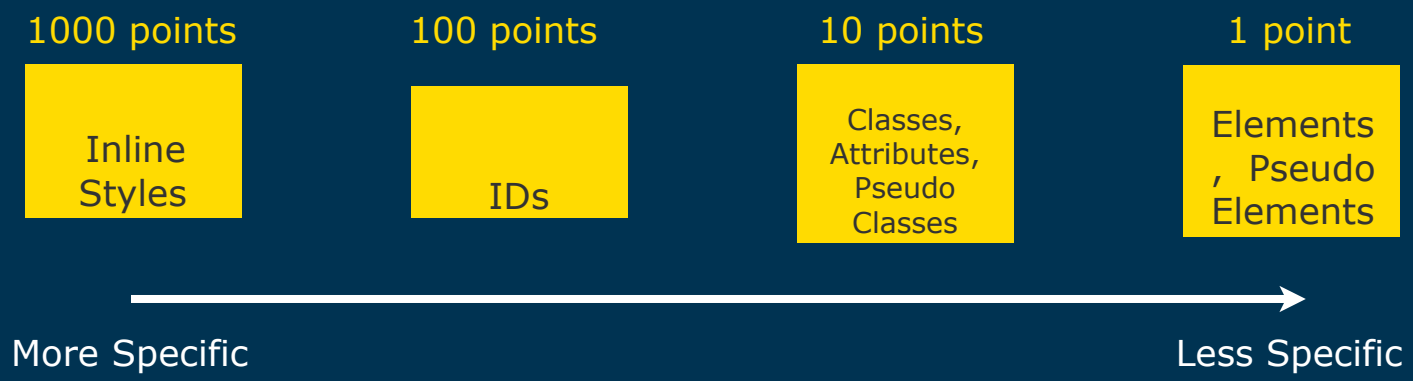
Usually, you won't add identical selectors in your CSS, however, the more complicated the document, the higher the chance of your rules conflicting with each other. Look at this:

```
p em { font--weight:bold; }
em { font--weight:normal; }
```

Even though the single em style came later in the document, it's font-weight will be bold, simply because the p em selector is more specific than em alone.

Specificity Calculator

There is actually a way to calculate the specificity of a selector to see if it will take precedence over another. For example:



For Example

`h1` = Specificity of 1

Because it’s a single HTML element.

For Example

.bass = Specificity of 10

Because it’s a class selector.

For Example

#fish h1.bass em = Specificity of 112

Because it’s an ID + HTML Element + Class + HTML Element.

100 1 10 1

Another Example

If you declared this in your CSS:

```
111 points #fish h1.bass { background: blue;}
110 points   #fish .bass { background: red; }
```

The first selector would win, and the background of .bass would be blue because it has more points than the bottom selector, even though the second selector comes later in the stylesheet.

Fun Resource!

There is a great resource on the web called S**pecificity Calculator** which allows you to type two groups of selectors and it will automatically calculate the specificity of each selector for you. Then you can "sort by specificity" to see which selector's styles would be applied.

The URL is: http://specificity.keegan.st/